Energy-Efficient Routing for a Neighbourhood Rooftop Water-Tank Monitoring Network: A Software Platform, a Step-by-Step Tutorial, and an Evaluation of LEACH, PEGASIS and Improved Chain Protocols on Real Building Data

IBRAHIM ABUZANT[1], **JEHAD M. HAMAMREH**[2] **(Supervisor)**

[1]*Department of Computer Science, An-Najah National University, Nablus, Palestine*
[2]*Department of Computer Science and Cyber Security, Faculty of Information Technology and Computer Science, An-Najah National University, Nablus, Palestine*
Corresponding author: Ibrahim Abuzant (e-mail: s12114660@najah.edu)

ABSTRACT Many households in Palestinian cities depend on rooftop water tanks, and the occupants cannot tell when a tank is about to run dry or overflow unless someone climbs up to look. A neighbourhood-scale wireless sensor network (WSN) of battery-powered tank-level nodes can report this automatically, but it is useful only while its batteries last, so how the nodes route their data decides the lifetime of the system. This report treats that small, local problem as a complete, reproducible case study. It delivers (i) a documented survey of the 14 repositories of the GitHub `leach` topic, none of which implements PEGASIS; (ii) a NumPy-only simulator of Direct transmission, LEACH and PEGASIS with nine groups of self-tests; (iii) a statistical comparison on 50 random deployments in which PEGASIS outlives LEACH by 47.6% (first node death), 95.1% (half of the nodes) and 34.6% (last node); and (iv) a diagnosis showing that PEGASIS' first-node death is almost entirely determined by the longest link of its greedy chain (Spearman $\rho = -0.99$). We then study two chain-protocol ideas, a 2-opt (travelling-salesman heuristic) chain and an energy-aware leader, and a multi-chain variant that cuts the chain at long links. A full $2 \times 2 \times 2$ ablation over 30 random layouts shows that the 2-opt chain and the energy-aware leader produce the gains, whereas cutting the chain *hurts* first- and half-node death. The best variant, PEGASIS-E, raises first-node death from 38 ± 13 to 569 ± 43 rounds and half-node death from 351 ± 27 to 959 ± 15 rounds ($p < 0.001$) on 80 nodes in a 400 m block, and on a real map of 80 buildings from OpenStreetMap in Nablus it raises first-node death from 4 to 677 rounds. The work is delivered as a software platform (Python engine, REST API, SQLite experiment store, web dashboard, Node-RED flow, 20 automated tests) and written as a tutorial so that every number and figure can be reproduced. The individual ideas are known from the PEGASIS literature (Section 10); the contribution is the controlled evaluation, the real-data study and the platform.

INDEX TERMS Wireless sensor networks, routing, LEACH, PEGASIS, network lifetime, energy efficiency, water-tank monitoring, simulation, reproducible research.

1. INTRODUCTION

1.1 The local problem

In the West Bank, where Nablus is located, rooftop water tanks are the normal answer to an unreliable supply: B'Tselem reports that 92% of Palestinians in the West Bank store water in tanks on their roofs and that only 36% receive running water every day [15]. At the national level, the Palestinian Central Bureau of Statistics and the Palestinian Water Authority report that average Palestinian water consumption per capita remains below the 100 litres per day recommended as a minimum by the World Health Organization [16]. A tank that runs empty means a household without water; a tank that overflows wastes water. Today the only "sensor" is a person looking at the tank. The problem we pick is deliberately small: *a single residential block of about one hundred buildings, each with a rooftop tank, needs to report the tank levels to one collection point (a base station, BS) for as long as possible without anyone replacing batteries.* The scale is a neighbourhood, not a country; the value is that the whole problem can be simulated, understood and reproduced by one student on a laptop.

1.2 Why routing decides lifetime

A wireless sensor node spends most of its energy on its radio. The energy needed to transmit a bit grows with the *square* of the distance in free space and with the *fourth power* once the distance exceeds a crossover value d_0 (about 88 m in our model, Section 4). Sending every reading straight to a distant BS therefore drains the far nodes quickly, while sending through neighbours spreads the cost. How the data is relayed is the *routing protocol*. Two classic energy-aware protocols are:

- **LEACH** (Low-Energy Adaptive Clustering Hierarchy) [1, 2]: nodes randomly take turns as cluster heads (CHs); members send to the nearest CH, the CH fuses the data and sends one packet to the BS.
- **PEGASIS** (Power-Efficient GAthering in Sensor Information Systems) [3]: nodes form a greedy chain; each node talks only to its nearest neighbour, data is fused hop by hop, and one rotating leader sends a single packet to the BS.

1.3 Contributions

1. A fully specified local use case, with every assumption listed (Table 2).
2. A documented survey of the GitHub `leach` topic (Section 3) and a justified reference choice.
3. A *NumPy-only* simulator of Direct, LEACH and PEGASIS with nine groups of self-tests, several of them checked against hand calculations (Section 6.4).
4. A statistical comparison on 50 paired deployments plus four sensitivity studies (sink position, network size, LEACH's p, layout) and a diagnosis of PEGASIS' high first-node-death variance.
5. Two chain-protocol ideas (2-opt chain, energy-aware leader), a multi-chain variant, a complete $2 \times 2 \times 2$ ablation

and an honest negative result (Sections 10–11).
6. A pipeline from OpenStreetMap building footprints to a real layout, and an evaluation on 80 real buildings in Nablus (Section 9).
7. A software platform: HTTP/JSON API, SQLite experiment store, web dashboard, Node-RED flow, a parallel experiment runner, and 20 automated tests (Section 8).
8. A complete tutorial (Section 6) and appendices with the code, so the work can be repeated step by step.

1.4 Organisation

Section 2 gives the background, Section 3 the repository survey, Section 4 the system model, Section 5 the protocols, Section 6 the tutorial, Section 7 the baseline results, Sections 8–12 the software platform, the real building data, the improved protocols, the ablation study and a discussion of Sionna, Section 13 the limitations, and Section 14 concludes.

2. BACKGROUND AND RELATED WORK

Wireless sensor networks consist of many small, battery-powered nodes that sense the environment and forward data to a sink [4]. Because radio communication dominates the energy budget, a large body of work studies energy-aware routing; surveys such as [5] classify protocols as flat, hierarchical (clustering) or location-based. LEACH is the best-known hierarchical protocol; its key ideas are randomised, rotating cluster heads (to share the heavy CH load) and local data fusion (to reduce the traffic towards the BS) [2]. PEGASIS observed that LEACH still pays for cluster set-up and for long member-to-CH and CH-to-BS links, and replaces clusters with a chain so that every transmission is as short as possible and only one node per round talks to the BS [3].

The usual lifetime metrics are **FND** (round of the *first node death*), **HND** (round when *half* of the nodes are dead) and **LND** (round of the *last node death*). FND approximates the time until coverage first degrades; HND and LND describe how gracefully the network fades.

The format of this report follows the supervisor's tutorial-style paper on building 5G networks with open-source software [6]: a motivated problem, background, a step-by-step "how to" with commands and outputs, hardware/software choices and a conclusion. Here the "hardware" is a laptop and the "software" is Python.

3. SURVEY OF THE GITHUB `LEACH` TOPIC

The supervisor asked us to go through the repositories listed at `https://github.com/topics/leach`. On 10 October 2026 the topic listed 14 public repositories (Table 1). We read every listing entry (language, stars, last update, description, topics) and read the full README of the Python repository `Nachi28/WSN_LEACH`. **We did not execute the code of any repository**; the selection below is therefore documentation-based, and our own simulator is an independent re-implementation validated by hand-computed tests (Section 6.4).

3.1 Selection criteria

Our project needs a code base that is (C1) free to run without licensed software, (C2) based on the standard first-order radio model so that results are comparable with the literature, (C3) documented well enough for a beginner, and (C4) easy to extend with a second protocol.

3.2 Findings

- **No repository in the topic mentions PEGASIS.** Neither the descriptions nor the topic tags of the 14 entries do, so the supervisor's requirement of comparing LEACH and PEGASIS cannot be met by simply cloning one of them.
- Six repositories are MATLAB-only (C1: needs a commercial licence or Octave compatibility checks), one needs NS-2/Tcl, one OMNeT++/INET, one Cooja/Contiki-NG, one Java; these are heavy tool-chains for a beginner (C3).
- Of the four Python repositories, `PyNetSim` implements a reinforcement-learning variant (LEACH-RLC) that goes well beyond the baseline; `I-LEACH-PY` has only a one-line listing description; `forest-fire-dashboard` is a hardware/browser demonstrator.
- `Nachi28/WSN_LEACH` is the best fit: Python with only NumPy/Matplotlib/pandas/openpyxl, a README that documents the classes `WSN`, `Node` and `Leach` and explicitly uses the Heinzelman constants (`ETX`, `ERX`, `Efs`, `Emp`, `EDA`, `do`), cluster-head selection by threshold probability, and an energy-based termination criterion. Its limitations for our purpose are that it has no PEGASIS, that its workflow requires manually editing an Excel file path inside two scripts, and that its outputs are PDR and delay rather than FND/HND/LND.

3.3 Decision

We adopt `Nachi28/WSN_LEACH` as the *reference design* (same radio model and constant names) and implement a new, self-contained simulator that adds PEGASIS, the Direct baseline, FND/HND/LND metrics, statistics and tests. This is a deliberate choice: copying a repository whose behaviour we have not verified would not satisfy a "works perfectly" requirement, whereas a small simulator with hand-verified tests can be audited line by line (Appendix A).

4. SYSTEM MODEL AND PROBLEM STATEMENT

4.1 Scenario

A residential block is modelled as a $200\,\mathrm{m} \times 200\,\mathrm{m}$ square containing $N = 100$ static nodes, one per building (rooftop tank). Positions are drawn uniformly at random (the standard in the LEACH literature) or from a jittered street grid. The BS (a community collection point with mains power) is located at $(100, 250)$ m, 50 m beyond the northern edge of the block. One *round* is one reporting period in which every living node produces one 4000-bit reading that must reach the BS. **The deployment is synthetic;** Section 6 explains how to replace it by surveyed coordinates of real tanks.

4.2 First-order radio model

We use the model of Heinzelman et al. [2]. Transmitting k bits over distance d costs

$$E_{\mathrm{Tx}}(k,d) = \begin{cases} kE_{\mathrm{elec}} + k\,\varepsilon_{fs}\,d^2, & d < d_0 \\ kE_{\mathrm{elec}} + k\,\varepsilon_{mp}\,d^4, & d \geq d_0 \end{cases} \tag{1}$$

with crossover distance

$$d_0 = \sqrt{\varepsilon_{fs}/\varepsilon_{mp}} \approx 87.7\ \mathrm{m}. \tag{2}$$

Receiving costs $E_{\mathrm{Rx}}(k) = kE_{\mathrm{elec}}$ and fusing $m \geq 2$ packets into one costs mkE_{DA}. *Worked example:* a 4000-bit packet

Table 1. Repositories listed under the GitHub topic `leach` (10 Oct. 2026; stars, dates and focus taken from the listing page).

#	Repository	Language	Stars	Last update	Focus (from listing)
1	`Rajdeep-Das/WSN_LEECH_Protocol`	MATLAB	33	Mar 2020	LEACH implementation
2	`omarfarabi/Leach_Heed_Comparative_analysis`	MATLAB	19	Sep 2019	LEACH vs. HEED
3	`HritwikSinghal/I-LEACH`	MATLAB	8	May 2021	Improved LEACH (circular patches)
4	`xcodeBn/PiLeachProtocol`	C++	7	Sep 2026	LEACH in OMNeT++/INET
5	`mbfakourii/Leach-NS2`	Tcl	6	Sep 2021	LEACH + AODV in NS2
6	`liagkos/leach`	Java	5	Mar 2018	LEACH in Java
7	`Laraib-Azmat/MLEACH`	MATLAB	3	Feb 2024	Mobility in LEACH
8	`fdojurado/PyNetSim`	Python	3	Feb 2026	LEACH-RLC (reinforcement learning)
9	`Laraib-Azmat/Leach_Protocol`	MATLAB	3	Feb 2024	LEACH MAC protocol
10	`kanika2296/distributed-query-processing-in-wsn`	MATLAB	2	Nov 2020	Distributed query processing
11	`Nachi28/WSN_LEACH`	Python	1	Jun 2024	LEACH, PDR and delay, Excel output
12	`HritwikSinghal/I-LEACH-PY`	Python	1	May 2021	Python port of I-LEACH
13	`poe125/LEACH_contikiNG`	Makefile	0	Nov 2025	LEACH and LEACH-C in Cooja
14	`nishankswamy/forest-fire-dashboard`	Python	0	Aug 2026	LoRa testbed + browser simulator

Table 2. Simulation parameters (all defined in class `Params`).

Parameter	Symbol	Value
Number of nodes	N	100
Block size	L	200×200 m
Base station	BS	$(100, 250)$ m
Initial energy / node	E_0	0.5 J (normalised)
Electronics energy	E_{elec}	50 nJ/bit
Free-space amplifier	ε_{fs}	10 pJ/bit/m^2
Multipath amplifier	ε_{mp}	0.0013 pJ/bit/m^4
Fusion energy	E_{DA}	5 nJ/bit/signal
Crossover distance	d_0	87.7 m
Data packet	k	4000 bit
Control packet	k_c	200 bit
LEACH CH fraction	p	0.05

costs 0.30 mJ over 50 m, 0.72 mJ over 100 m, but 20.5 mJ over 250 m. The last number is why direct transmission from far nodes is ruinous: it is 68 times the cost at 50 m. Receiving the same packet costs 0.20 mJ and fusing one extra signal 0.02 mJ.

4.3 Metrics

FND, **HND** and **LND** (Section 2) in rounds; the **number of sequential transmissions per round** (a latency proxy); the **packets that reach the BS per round**; and the total number of sensor readings represented at the BS during the lifetime.

4.4 Problem statement

Given the scenario above, which routing protocol (Direct, LEACH or PEGASIS) maximises FND, HND and LND, at what latency cost, and how robust is the answer to the BS position, network size, LEACH's p and the building layout?

5. PROTOCOLS

5.1 Direct transmission (baseline)

Every node sends its packet straight to the BS. The lifetime of node i at distance d_i is, analytically, $\lceil E_0 / E_{\text{Tx}}(k, d_i) \rceil$ rounds. This closed form is used as a test of the simulator (T2, Section 6.4).

5.2 LEACH

At the start of round r, each node n that has not yet been a CH in the current epoch of $1/p$ rounds becomes a CH with probability

$$T(n) = \frac{p}{1 - p\left(r \bmod \frac{1}{p}\right)}. \tag{3}$$

In the last round of an epoch $T(n) = 1$, so every node serves exactly once per epoch. Algorithm 1 lists one round, including the set-up messages (ADV, JOIN, SCHEDULE) that earlier simple implementations often omit. A CH with m members spends

$$E_{CH} = m E_{\text{Rx}}(k) + (m+1)\, k E_{\text{DA}} + E_{\text{Tx}}(k, d_{BS}) \tag{4}$$

per steady-state round, plus control energy; a member spends $E_{\text{Tx}}(k, d_{CH})$.

Algorithm 1 One LEACH round

1: if $r \bmod (1/p) = 0$: mark all living nodes eligible
2: each eligible living node becomes CH with prob. $T(n)$ from (3); CHs become ineligible
3: **set-up:** each CH broadcasts ADV reaching the farthest living node; every other node receives all ADVs, sends JOIN to the nearest CH; each CH broadcasts a TDMA schedule to its members
4: **steady state:** members send data to their CH; CH receives, fuses own+member data, sends one packet to the BS
5: if no CH was elected: all nodes send directly to the BS this round
6: nodes with energy ≤ 0 die

5.3 PEGASIS

Chain construction. Starting from the node farthest from the BS, repeatedly append the closest node that is not yet on the chain; the chain is rebuilt only when a node dies. **Leader rotation.** In round r the leader is the $(r \bmod n)$-th living node, where n is the number of living nodes, so all nodes serve equally often. **Gathering.** Data flows from both chain ends towards the leader; each relay receives, fuses its own reading with the incoming one (cost $2kE_{\text{DA}}$) and forwards. The leader receives from up to two neighbours, fuses, and sends *one* packet to the BS (Algorithm 2). With the leader at chain position L of n nodes, the number of sequential transmissions is $\max(L, n-1-L) + 1$.

5.4 Qualitative trade-offs

LEACH is *distributed*: a node only needs to know p and its distance to the CHs, so it works without global knowledge, but

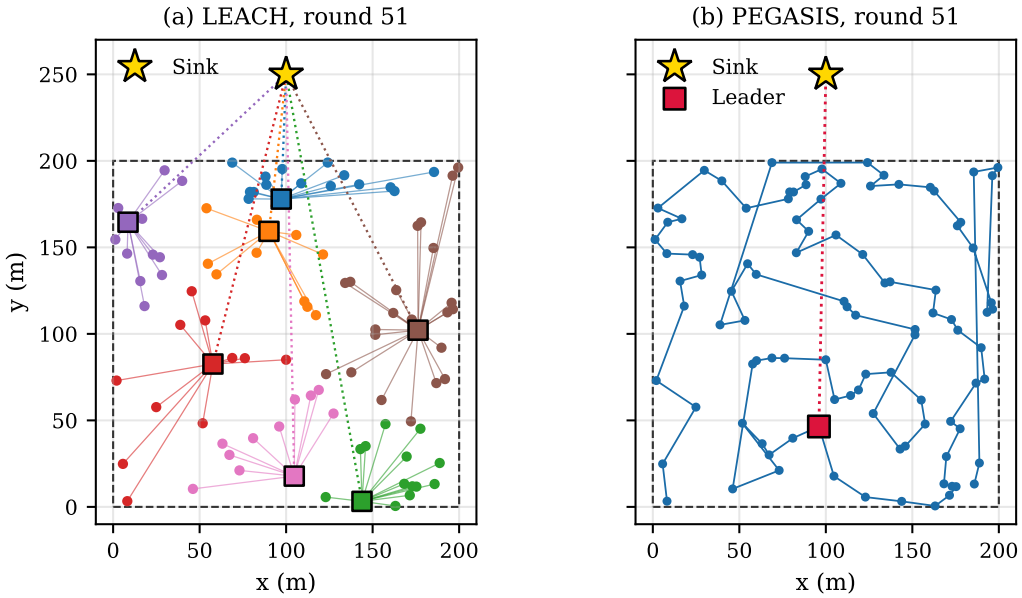


Figure 1. One round on the same random deployment ($N = 100$). (a) LEACH: squares are cluster heads, thin lines join members to their CH, dotted lines are CH→BS links. (b) PEGASIS: the greedy chain; the red square is the leader, the dotted line its single link to the BS (star).

Algorithm 2 One PEGASIS round

1: if a node died since the last round: rebuild greedy chain from the node farthest from the BS
2: leader ← living node number $(r \bmod n)$; $L \leftarrow$ its chain position
3: nodes $0..L-1$ pass data towards the leader: i sends to $i+1$; relays receive and fuse
4: nodes $n-1..L+1$ pass data towards the leader symmetrically
5: leader receives, fuses (up to 3 signals), sends one packet to the BS
6: nodes with energy ≤ 0 die

it wastes energy on set-up and long CH-to-BS links. PEGASIS needs the node positions (or a chain-building procedure) and gives every node only short neighbour links, but relaying data along an n-node chain takes $O(n)$ sequential transmissions, so its latency grows with the network.

6. TUTORIAL: HOW TO REPRODUCE EVERYTHING

This section is written for a reader who has never simulated a sensor network. We first answer the five basic questions and then give the steps.

6.1 The five questions

What? A simulation that counts the radio energy of every node, round after round, under three routing protocols, and records when nodes die.
Why? Because a field trial of 100 battery nodes would take months and money; a simulator gives the same *relative* comparison in about two minutes, and every run is repeatable.
Where? Anywhere with Python: a laptop. The write-up is compiled in Overleaf. The *modelled* place is a Nablus residential block.
When? Use LEACH or PEGASIS-type protocols when nodes are battery-powered, static, and a delay of up to a reporting period is acceptable; tank levels change slowly, so latency is not critical here.
How? Steps 1–9 below.

6.2 Tools

Python ≥3.10 (we used 3.12.3) with **NumPy** [8] 2.4.4, **SciPy** [9] 1.17.1 and **Matplotlib** [10] 3.10.8; a text editor or VS Code; **Overleaf** with the pdfLaTeX compiler for the report. No GPU, no paid software, no network access is required to run the simulator.

6.3 Steps

Step 1 – Define the real problem and measure it. Decide the block, the number of tanks and the BS location. For a real study, record the latitude/longitude of every tank (a phone GPS or Google Earth), convert to metres relative to one corner, and replace `make_layout` by loading that list. This is the one place where local data can make the work genuinely Nablus-specific.

Step 2 – Create the project folders.

```
mkdir wsn_report && cd wsn_report
mkdir code figures results
```

Step 3 – Install the libraries.

```
python -m pip install numpy scipy matplotlib
```

Step 4 – Write the radio model (`e_tx`, `e_rx`, `e_fuse` in `wsn_sim.py`) exactly as in Eq. (1). Put *all* constants in one `Params` class so that every assumption is visible and changeable.

Step 5 – Implement the baseline and LEACH (`run_direct`, `run_leach`): one loop over rounds, subtract energy, mark dead nodes, stop when none is alive.

Step 6 – Implement PEGASIS (`build_chain`, `run_pegasis`) following Algorithm 2.

Step 7 – Verify before trusting.

```
cd code
python wsn_sim.py
```

Table 3. Self-tests executed by `python wsn_sim.py` (all pass).

ID	What is checked
T1	Free-space and multipath curves meet at $d_0 = 87.71$ m
T2	Direct: simulated death round of each node equals $\lceil E_0/E_{\mathrm{Tx}} \rceil$
T3	PEGASIS: 3-node, 1-round energies equal the hand calculation
T4	LEACH and PEGASIS: residual energy and alive count never increase
T5	Direct: the node farthest from the BS dies first
T6	2-opt chain: same node set, link energy never above the greedy chain
T7	PEGASIS-MC: 2 chains, one round equals the hand calculation
T8	Generic variant runner reproduces PEGASIS and PEGASIS-2opt exactly
T9	Retries and sleep energy shorten lifetime; defaults change nothing

Expected output: ten lines ending in `[PASS]` and `All self-tests passed.` (Table 3). Do not continue if any test fails.

Step 8 – Run the experiments.

```
python run_experiments.py
```

This writes LaTeX tables to `results/`, figures (PDF and PNG) to `figures/`, and `summary.json` and `baseline_raw.csv` for traceability. It takes about one to two minutes.

Step 9 – Compile the report. Upload the folder (`main.tex`, `code/`, `figures/`, `results/`) to Overleaf, choose pdfLaTeX, and press Recompile. The tables are `\input` from `results/`, so *no number in this paper is typed by hand.*

6.4 Verification

Table 3 lists the self-tests. T3 is the most important: three nodes on a line, one PEGASIS round, compared with an independent hand calculation of every transmit, receive and fusion term; the simulated and hand-computed energies agree to numerical precision.

6.5 Experimental protocol

The baseline uses 50 random deployments (seeds 0–49); each deployment is run under all three protocols (paired design); LEACH's random draws use seed 10000+deployment seed. Sensitivity studies use 30 deployments. The baseline rows of the sensitivity tables (e.g. $N = 100$, BS $y = 250$ m, $p = 0.05$) therefore differ slightly from Table 4, which uses 50 deployments. We report means with 95% Student-t confidence intervals and use the paired two-sided Wilcoxon signed-rank test [7] for LEACH vs. PEGASIS.

7. RESULTS AND DISCUSSION

7.1 Baseline scenario

Table 4 and Figs. 2–4 summarise the baseline. **Direct** transmission loses its first node after only 22 ± 1 rounds and half of the network after 137 ± 8, because far nodes pay the d^4 term. Paradoxically its LND is the longest (1506 ± 26), since the few nodes close to the BS keep transmitting cheaply; LND alone is therefore a misleading metric and must be read together with FND/HND. **LEACH** lifts FND to 206 ± 7 and

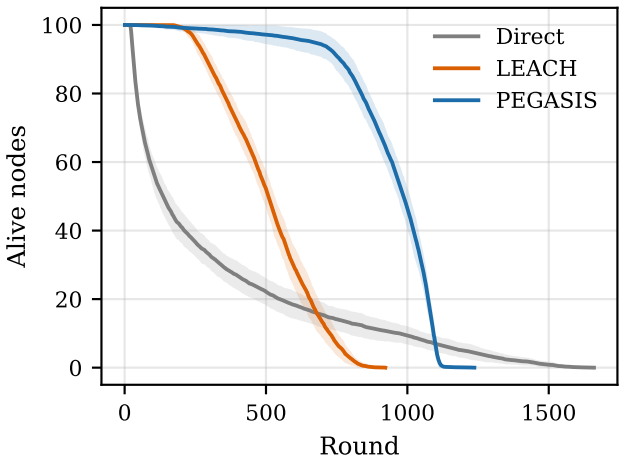


Figure 2. Alive nodes vs. round (mean of 50 deployments; shaded: ± 1 s.d.).

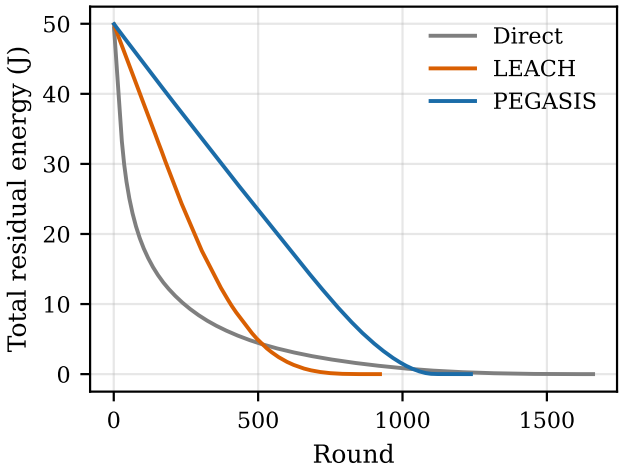


Figure 3. Total residual network energy vs. round.

HND to 505 ± 7, but all nodes are dead by 838 ± 10. **PEGASIS** reaches 304 ± 60, 986 ± 6 and 1129 ± 8 rounds, and its alive-node curve stays high for most of the lifetime and then drops steeply (Fig. 2): PEGASIS needs only 143 rounds to go from half the nodes dead (HND) to none alive (LND), LEACH needs 333. Because the load is evenly shared, PEGASIS nodes die at nearly the same time.

7.2 Statistical comparison

Table 5 shows that PEGASIS is significantly longer-lived on all three metrics. The FND difference ($p = 0.014$) is the weakest because PEGASIS' FND varies strongly between deployments; it stays below a Bonferroni threshold of $0.05/3 \approx 0.017$ but only narrowly, so we describe the FND advantage as moderate and the HND/LND advantages as strong.

7.3 Why is the PEGASIS first-node-death so variable?

The confidence interval of PEGASIS' FND (± 60) is ten times larger than LEACH's (± 7). The greedy chain sometimes has to close with a long link back to a far-away leftover node. In 92% of our 50 layouts the initial chain contains at least one link longer than $d_0 = 87.7$ m (on average 1.56 such links; the longest link observed is 241 m). A node on such a link pays

Table 4. Baseline results over 50 random deployments (mean ± 95% CI, rounds). Hops/round: sequential transmissions per round; Sink pkts/round: packets reaching the BS per round (lifetime average); Readings: sensor readings represented at the BS over the whole lifetime.

Protocol	FND	HND	LND	Hops/round	Sink pkts/round	Readings ($\times 10^4$)
Direct	22 ± 1	137 ± 8	1506 ± 26	1.0	20.84	3.1 ± 0.1
LEACH	206 ± 7	505 ± 7	838 ± 10	1.8	5.75	5.1 ± 0.1
PEGASIS	304 ± 60	986 ± 6	1129 ± 8	63.2	1.00	9.4 ± 0.1

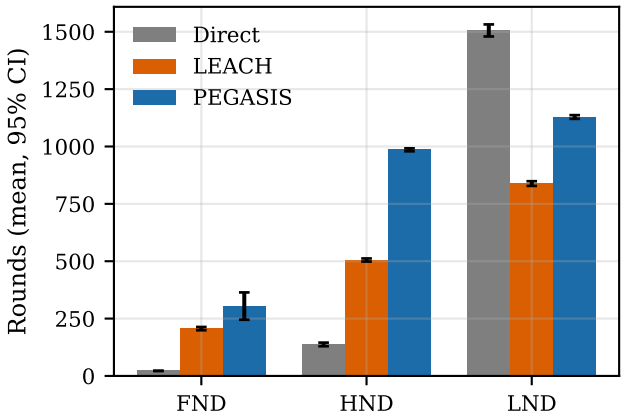


Figure 4. FND, HND and LND (mean ± 95% CI, 50 deployments).

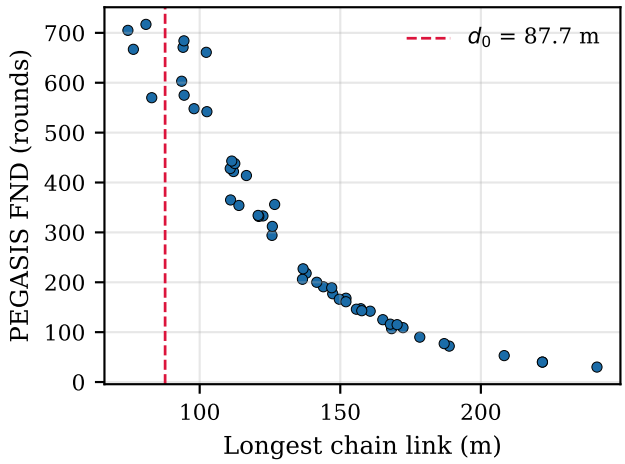


Figure 5. PEGASIS FND vs. the longest link of the initial chain (50 deployments).

Table 5. LEACH vs. PEGASIS, paired over 50 deployments.

Metric	LEACH mean	PEGASIS mean	PEGASIS vs LEACH	p (Wilcoxon)	Longer-lived
FND	206	304	+47.6%	0.014	PEGASIS
HND	505	986	+95.1%	$< 10^{-3}$	PEGASIS
LND	838	1129	+34.6%	$< 10^{-3}$	PEGASIS

the d^4 term every time it transmits across it. The Spearman rank correlation between the longest chain link and PEGASIS' FND is $\rho = -0.99$ ($p < 10^{-40}$; Fig. 5). In plain words: *in these simulations, how soon PEGASIS loses its first node is almost completely explained by how long its worst chain link is*. This is a correlation computed over 50 layouts; a causal test (for example constraining the chain so that no link exceeds d_0) is left as future work, and Section 11 tests ideas motivated by it directly, with a more nuanced outcome than this correlation suggests.

7.4 Latency and traffic at the sink

PEGASIS needs on average 63.2 sequential transmissions per round, against 1.8 for LEACH and 1 for Direct (Table 4), and delivers one packet per round to the BS (LEACH: 5.75 on average; Direct: 20.84). For tank levels reported once per period this is acceptable as long as the whole chain can finish inside the period; for alarms (e.g. overflow within seconds) it is not, and a hybrid or a separate alarm path would be required. The readings represented at the BS during the lifetime are 9.4×10^4 for PEGASIS, 5.1×10^4 for LEACH and 3.1×10^4 for Direct, i.e. PEGASIS delivers about 1.9 times as much data as LEACH per unit of initial energy.

7.5 Sensitivity to the BS position

Moving the BS away from the block (Table 6, Fig. 6) shortens lifetime for every protocol, but LEACH suffers more: at $y =$ 400 m LEACH's LND falls to 216 rounds whereas PEGASIS still reaches 723. The only case where LEACH wins on FND is the BS in the centre of the block ($y = 100$: 374 vs. 330); there the CH-to-BS links are short and PEGASIS' long-link penalty dominates its FND. PEGASIS is still ahead on HND (1093 vs. 675) and LND (1254 vs. 906) in that case. Because PEGASIS sends only one long-distance packet per round, its advantage grows with BS distance.

7.6 Sensitivity to network size

In Table 7 and Fig. 7, PEGASIS beats LEACH on HND and LND for every N from 25 to 200. For $N = 150$ the FND values are practically equal (248 vs. 253), and at $N = 25$ LEACH's FND is much lower (59 vs. 261); we did not investigate the cause. The price of PEGASIS grows linearly with N: from 13.1 to 133.8 sequential transmissions per round.

7.7 Sensitivity to LEACH's p

LEACH is sensitive to p (Table 8, Fig. 8): FND is best at the textbook value $p = 0.05$ (206), HND peaks near $p = 0.10$ (520) and LND keeps increasing up to $p = 0.20$ (980). No setting of p in the tested range reaches PEGASIS' reference values on any metric (30 deployments: FND 342, HND 988, LND 1126), so the PEGASIS advantage is not an artefact of a badly tuned LEACH.

7.8 Sensitivity to the layout

Replacing the uniform random placement by a jittered street grid (Table 9) changes the conclusions very little: LEACH 202/502/831 and PEGASIS 314/980/1125 rounds (FND/HND/LND), so the ranking does not depend on the placement model we chose.

Table 6. Mean lifetime (rounds, 30 deployments) vs. BS y-coordinate ($x = 100$ m; the block spans $y \in [0, 200]$).

	FND		HND		LND	
Sink y (m)	LEACH	PEGASIS	LEACH	PEGASIS	LEACH	PEGASIS
100	374	330	675	1093	906	1254
200	293	358	612	1054	880	1154
250	206	342	506	988	840	1126
300	137	282	364	856	664	1078
400	55	167	138	502	216	723

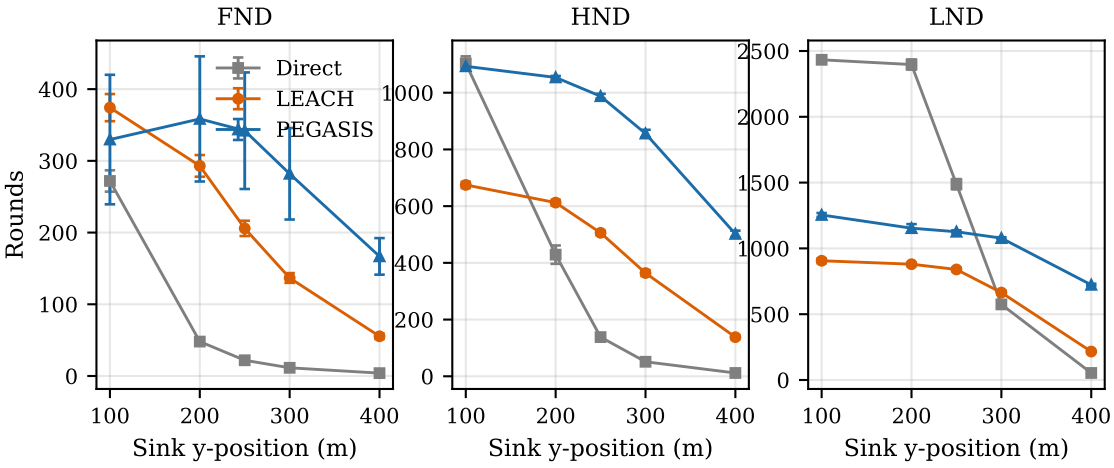


Figure 6. Sensitivity to the BS y-position (mean ± 95% CI, 30 deployments).

7.9 Answer to the problem statement

For this neighbourhood scenario PEGASIS gives the longest-lived network on HND and LND in every configuration we tested, and on FND in every case except the centred BS (and a practical tie at $N = 150$), at the price of a latency that grows with N. LEACH is the better choice only if nodes cannot know each other's positions, or if per-round latency must stay near two hops.

8. SOFTWARE PLATFORM

To make the study usable beyond a PDF, the simulator is wrapped in a small platform. Every component is open and runs on a laptop.

- **Engine** (`wsn_sim.py`, `platform_core.py`): the simulator and a *tank model*. Synthetic tank levels follow four supply zones, each served for 8 hours every 72 hours (+5 %/h), with household consumption of 0.25–0.90 %/h per tank. Alerts are raised at ≤ 15 % (low) and ≥ 99.5 % (overflow). A dead node is a tank nobody monitors any more. All of this is synthetic.
- **HTTP/JSON API** (`server.py`, standard library only): stateless endpoints `/api/summary`, `/api/state`, `/api/compare`, `/api/history`, `/api/curve`; invalid input returns HTTP 400/404 and never crashes the server, and a browser that refreshes or closes the tab in the middle of a request (connection aborted or reset) is handled silently, without a traceback.
- **SQLite experiment store** (`store.py`): tables `runs` and `curves`; every simulated run is stored (`/api/summary` saves the selected protocol, `/api/compare` all six); all SQL uses parameterised queries (tested with an injection string).
- **Dashboard** (one HTML file, no libraries): map with energy colours, cluster heads/leaders and links, alive-node chart, alerts, saved runs.
- **Node-RED flow** [13] (core nodes only): Flow 1 serves the dashboard and forwards `/wsn/api/*` to the engine, with a guard that answers each browser request exactly once; Flow 2 advances a simulated clock and emits only *new* tank alerts. The same dashboard page can be opened directly from the engine (port 8000) or through Node-RED (`/wsn` on port 1880); because the page calls the API with relative addresses, the browser only talks to the origin that served the page and no cross-origin (CORS) request occurs. Flow 2 was executed in Node-RED v5.0.9 on Windows and its debug panel showed the new-alert messages; the guard node was added after the first run produced `ERR_HTTP_HEADERS_SENT` errors.
- **Parallel experiment runner** (`ablation.py`): independent (variant, layout) jobs are distributed with `ProcessPoolExecutor`; an automated test checks that the parallel and sequential results are identical.
- **Tests**: 20 automated tests (simulator regression, tank model, state invariants, API, SQL injection, parallel equality, dashboard JavaScript run under a fake DOM for both URL layouts, an emulation of the Node-RED gateway for every endpoint the dashboard calls, silent handling of client disconnects, Node-RED function nodes run under a mocked runtime against the real engine), plus the simulator self-tests of Table 3. What these tests *cannot* check is how the dashboard looks in a browser.

Table 7. Mean lifetime (rounds, 30 deployments) vs. number of nodes in the same 200 m block; last column: mean sequential transmissions per PEGASIS round.

	FND		HND		LND		PEGASIS
N	LEACH	PEGASIS	LEACH	PEGASIS	LEACH	PEGASIS	hops/round
25	59	261	233	688	909	1013	13.1
50	107	298	319	872	823	1080	29.6
100	206	342	506	988	840	1126	63.5
150	248	253	600	1038	912	1134	99.7
200	256	329	628	1063	942	1161	133.8

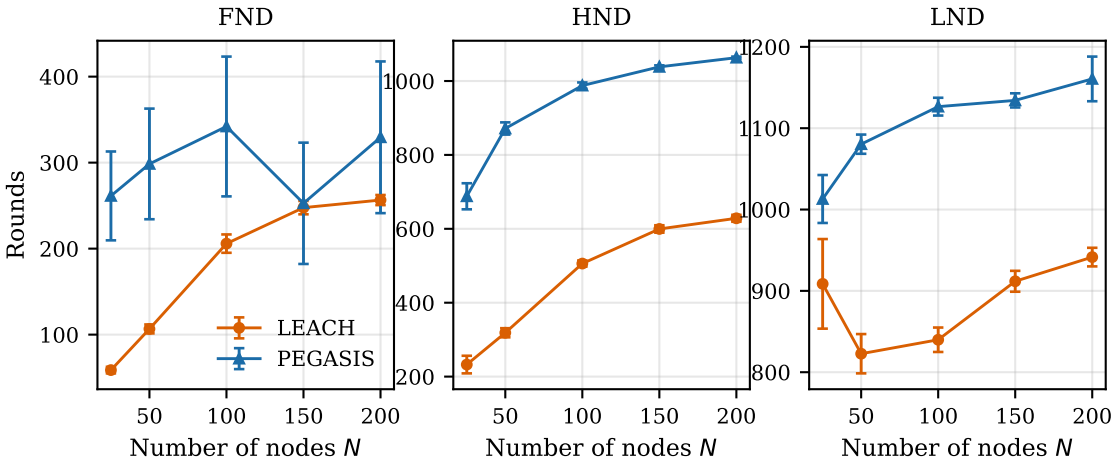


Figure 7. Sensitivity to the number of nodes (mean ± 95% CI, 30 deployments).

Table 8. LEACH lifetime (rounds, mean ± 95% CI, 30 deployments) vs. CH fraction p.

p	FND	HND	LND
0.02	89 ± 5	286 ± 10	873 ± 23
0.05	206 ± 11	506 ± 8	840 ± 15
0.10	166 ± 5	520 ± 15	938 ± 15
0.15	125 ± 4	454 ± 17	958 ± 15
0.20	93 ± 3	383 ± 17	980 ± 21

Table 9. Random vs. jittered street-grid layouts (30 deployments).

Layout	Protocol	FND	HND	LND
Uniform random	LEACH	206 ± 11	506 ± 8	840 ± 15
Uniform random	PEGASIS	342 ± 81	988 ± 8	1126 ± 11
Jittered street grid	LEACH	202 ± 5	502 ± 4	831 ± 10
Jittered street grid	PEGASIS	314 ± 68	980 ± 5	1125 ± 20

9. REAL BUILDING DATA FROM OPENSTREETMAP

Random positions are replaced by real building footprints from OpenStreetMap [12]. Procedure: (1) in Overpass Turbo (`https://overpass-turbo.eu`) zoom to a neighbourhood and query all ways tagged `building` inside the map view (`out geom`); (2) export GeoJSON; (3) run `prepare_layout.py`, which drops footprints below 40 m^2 and non-dwelling tags, projects longitude/latitude to local metres ($x = (\lambda - \lambda_0)\cos\varphi_0 \cdot 111320$, $y = (\varphi - \varphi_0) \cdot 110574$), selects the densest 400 m window (at most 100 buildings) and places the BS 50 m north of the block.

Result. The Nablus export used here contained 83 footprints (78 tagged `yes`, 5 `apartments`); 80 lie in one window of 397 × 384 m. A first export of another area contained only 12 buildings, which shows that OpenStreetMap coverage varies a lot between areas and may be incomplete. **Assumptions:** one building = one rooftop tank (large buildings may have several), and the BS position is chosen, not surveyed. The block is twice as wide as the synthetic 200 m block, so many chain links exceed $d_0 = 87.7$ m.

10. IMPROVED CHAIN PROTOCOLS

10.1 2-opt chain

A PEGASIS chain is an open Hamiltonian path through the nodes. The greedy rule (Section 5) is a nearest-neighbour heuristic for the travelling-salesman problem and can leave a long "closing" link. We minimise the total link *energy* $\sum_i E_{\mathrm{Tx}}(k, d_i)$ instead. Starting from the greedy chain we apply the classical 2-opt local search [11]: for chain positions $i < j$, reversing the segment between them replaces the links (a,b) and (c,d) by (a,c) and (b,d), which is accepted if

$$E(a,b) + E(c,d) - E(a,c) - E(b,d) > 0, \tag{5}$$

where $E(u,v) = E_{\mathrm{Tx}}(k, d_{uv})$; passes are repeated until no move improves the chain (at most 30). One pass is $O(n^2)$; the chain is recomputed only when a node dies. On the real map the greedy chain has 5 links $\geq d_0$ and 161.3 mJ total link energy, the 2-opt chain has none and 21.6 mJ (Fig. 9).

10.2 Energy-aware leader

Round-robin leadership ignores that a leader far from the BS pays a large d^4 cost. We choose in every round the leader with

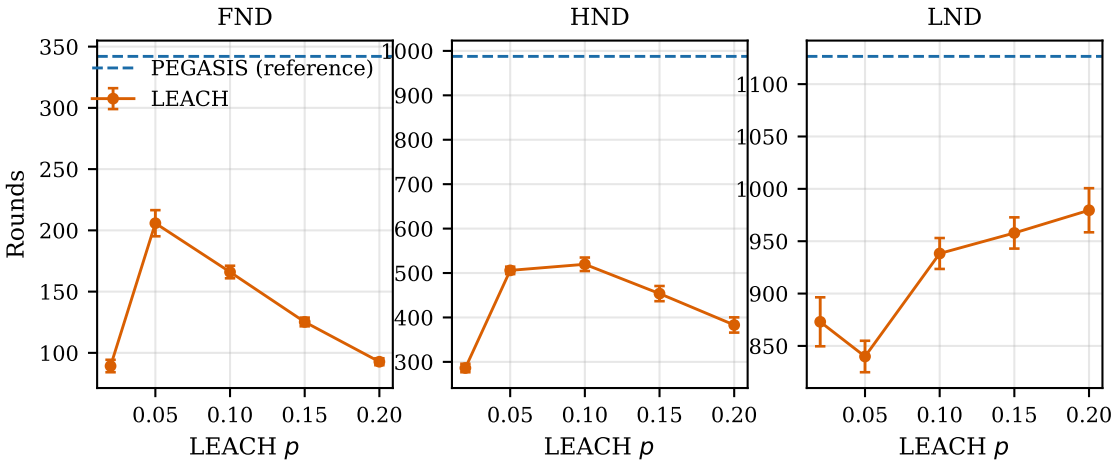


Figure 8. LEACH lifetime vs. p; the dashed line is PEGASIS at the baseline setting.

the largest

$$w_i = \frac{E_i}{E_{\mathrm{Tx}}(k, d_{i,BS})}, \tag{6}$$

the number of further rounds node i could afford as leader, so strong nodes close to the BS lead more often. This is essentially the leader weight $Q_i = E_i/D_i$ of IEEPB [18, 19]; the only difference is that we divide by the transmit energy instead of by the distance.

10.3 Variants

PEGASIS-2opt: 2-opt chain, round-robin leader. **PEGASIS-E**: 2-opt chain, energy-aware leader, one chain. **PEGASIS-MC** (multi-chain): 2-opt chain *cut* at every link $\geq d_0$; each piece elects its own energy-aware leader and every leader sends one packet to the BS.

10.4 Relation to the literature

None of these ideas is new. The long-link problem of the greedy chain and leader selection by residual energy and distance have been studied for PEGASIS-type protocols for years. EEPB [17] introduced a distance threshold when building the chain; IEEPB [18] removes the long-link problem with threshold computations and weights each node by residual energy and distance to the BS [19]. As summarised in [20], EECB also uses a distance threshold and chooses the leader from node-to-BS distances and residual energies, and Pegant replaces the greedy chain by an ant-colony search that minimises the total squared transmission distance, which is the same goal our 2-opt chain pursues with a different heuristic. Multi-chain variants with several leaders (MIEEPB [19]) and k-means clustering on top of IEEPB [21] have also been reported. Our designs are therefore *variations of known ideas*, written by us and *not* reproductions of those papers; we did not implement or run any published protocol, so this report makes no quantitative comparison with them. What it contributes is a controlled ablation of the three design choices, an evaluation on real building footprints, and a tested, reproducible platform.

11. ABLATION, REAL-MAP RESULTS AND REALISM

11.1 Ablation design

PEGASIS-MC changes three things at once, so every combination of the three switches (chain: greedy/2-opt; cut: no/yes; leader: rotate/energy) was run on the same 30 random layouts (80 nodes, 400 m block, BS at $(200, 450)$ m). The combination (greedy, no cut, rotate) is the original PEGASIS (test T8).

Table 10. Ablation on 30 random layouts (mean ± 95% CI, rounds).

Variant (chain/cut/leader)	FND	HND	LND	Hops/round
greedy/nocut/rotate	38 ± 13	351 ± 27	936 ± 46	26.5
greedy/nocut/energy	42 ± 19	880 ± 26	961 ± 19	45.7
greedy/cut/rotate	12 ± 5	239 ± 31	1043 ± 46	14.7
greedy/cut/energy	17 ± 7	505 ± 98	956 ± 31	23.8
2opt/nocut/rotate	112 ± 9	505 ± 22	1027 ± 37	31.3
2opt/nocut/energy	569 ± 43	959 ± 15	985 ± 16	58.9
2opt/cut/rotate	50 ± 15	413 ± 33	1071 ± 47	23.5
2opt/cut/energy	278 ± 107	791 ± 60	953 ± 26	42.2

Table 11. Main effect of each switch (average over the other two switches; paired Wilcoxon).

Switch turned on	Metric	Change	p (Wilcoxon)
2-opt chain (vs greedy)	FND	+828.7%	<0.001
2-opt chain (vs greedy)	HND	+35.1%	<0.001
2-opt chain (vs greedy)	LND	+3.6%	<0.001
cut long links (vs no cut)	FND	-53.0%	<0.001
cut long links (vs no cut)	HND	-27.7%	<0.001
cut long links (vs no cut)	LND	+2.9%	0.052
energy-aware leader (vs rotate)	FND	+325.5%	<0.001
energy-aware leader (vs rotate)	HND	+108.2%	<0.001
energy-aware leader (vs rotate)	LND	-5.5%	<0.001

Findings (Tables 10–11, Fig. 10). (1) The 2-opt chain is the strongest single change: first-node death +828.7 %, half-node death +35.1 %, last-node death +3.6 % (all $p < 0.001$). (2) The energy-aware leader raises FND by +325.5 % and HND by +108.2 %, but *lowers* LND by 5.5 % ($p < 0.001$): it protects the weak nodes early and drains the strong ones faster. (3) **Cutting the chain at long links does not help**: FND −53.0 % and HND −27.7 %; the LND change of +2.9 % is not significant at 5 % ($p = 0.052$). A plausible, *untested* explanation is that every piece needs its own transmission to the BS, so more nodes pay the long d^4 BS link. (4) The best variant is 2-opt / no cut / energy leader, i.e. PEGASIS-E: 569 ± 43 rounds to first death versus 38 ± 13 for the original, and 959 ± 15 versus 351 ± 27 to half-node death. This corrects our earlier expectation that cutting would be the main remedy.

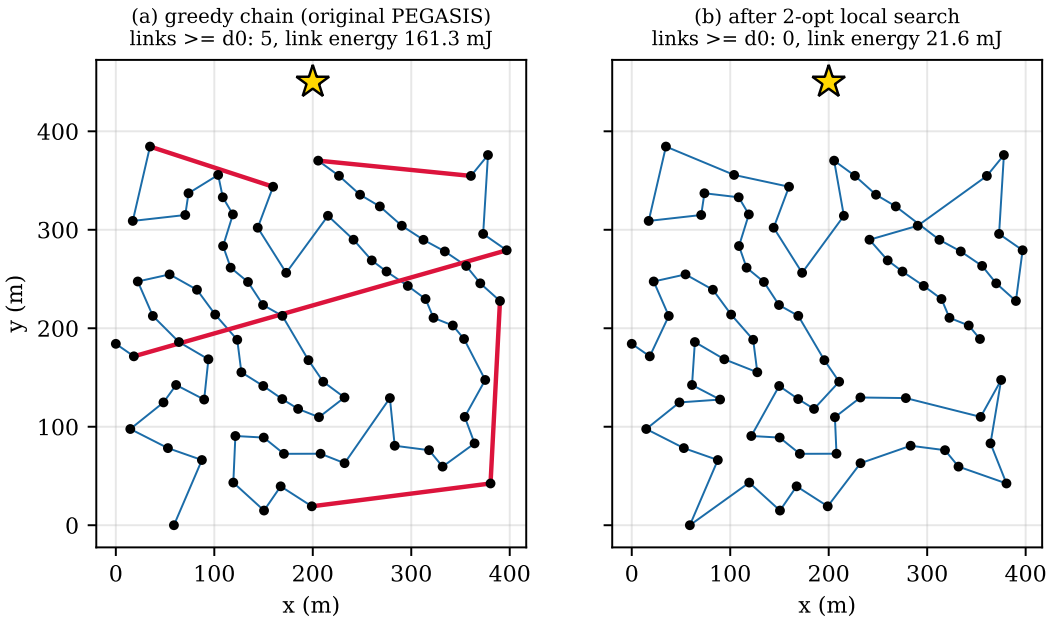


Figure 9. Real 80-building map. (a) Greedy PEGASIS chain: red links are longer than d_0. (b) The same nodes after 2-opt local search. The star is the BS.

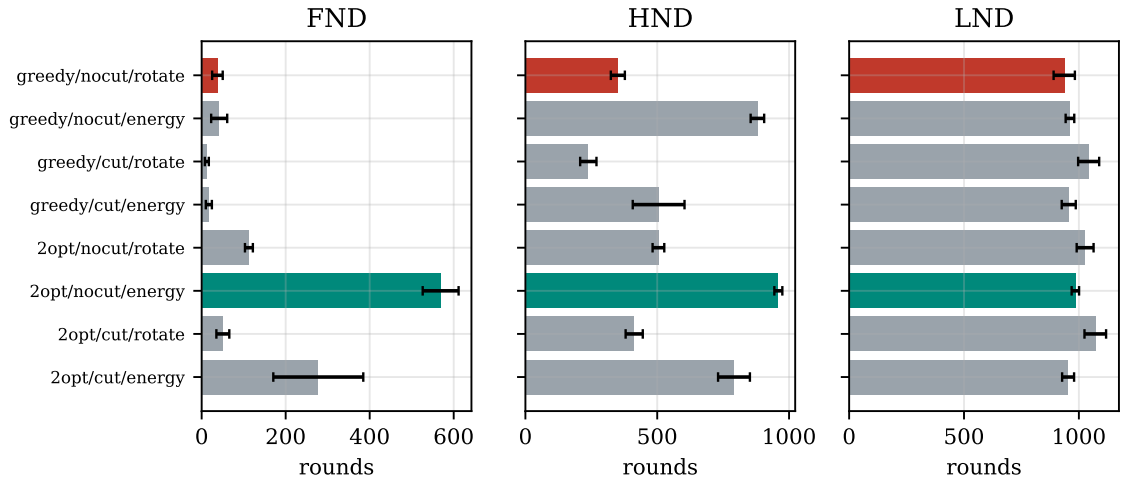


Figure 10. Ablation: lifetime of the eight variants (red: original PEGASIS; green: 2-opt chain, no cut, energy-aware leader = PEGASIS-E).

Table 12. All protocols, 30 random layouts (80 nodes, 400 m block).

Protocol	FND	HND	LND	Hops/round
Direct	3 ± 0	17 ± 2	1241 ± 96	1.0
LEACH	14 ± 2	57 ± 3	403 ± 57	1.4
PEGASIS	38 ± 13	351 ± 27	936 ± 46	26.5
PEGASIS-2opt	112 ± 9	505 ± 22	1027 ± 37	31.3
PEGASIS-E	569 ± 43	959 ± 15	985 ± 16	58.9
PEGASIS-MC	278 ± 107	791 ± 60	953 ± 26	42.2

11.2 All protocols on random and real layouts

In this larger block (Tables 12–14, Fig. 11) Direct transmission and LEACH die quickly because most links are in the d^4 regime, and LEACH is significantly worse than plain PEGASIS on all three metrics. On the real map PEGASIS-E keeps the first node alive for 677 rounds against 4 for PEGASIS and 16 for LEACH, and half of the nodes for 925 rounds against 339 and 66. Direct transmission has the longest last-node life (1097 rounds) because the few nodes next to the BS survive, which again shows that LND alone is misleading. The price of PEGASIS-E is latency: 58.9 sequential transmissions per round on random layouts versus 26.5 for the original and 1.4 for LEACH, which is acceptable only for slowly changing quantities such as tank levels. On the real map PEGASIS-E and PEGASIS-MC have the same FND (677), so we cannot tell from this single run whether they share the same bottleneck node.

11.3 Realism sensitivity

Retries and idle energy are added on the real map (Table 15 in the appendix): `attempts` multiplies every transmit and receive energy (mean transmissions per packet, ARQ), and `sleep` subtracts a constant energy per living node per round (the values 10^{-4} and $5 \cdot 10^{-4}$ J/round are comparable to the cost of receiving one packet; $E_0 = 0.5$ J). In all nine combinations PEGASIS-E has the largest FND and HND of the five

Table 13. PEGASIS variants and LEACH relative to plain PEGASIS (paired Wilcoxon, 30 layouts).

Protocol	Metric	Change vs PEGASIS	p
PEGASIS-2opt	FND	+199.5%	<0.001
PEGASIS-2opt	HND	+43.9%	<0.001
PEGASIS-2opt	LND	+9.7%	<0.001
PEGASIS-E	FND	+1414.5%	<0.001
PEGASIS-E	HND	+173.6%	<0.001
PEGASIS-E	LND	+5.2%	0.036
PEGASIS-MC	FND	+639.1%	<0.001
PEGASIS-MC	HND	+125.6%	<0.001
PEGASIS-MC	LND	+1.8%	0.797
LEACH	FND	-63.0%	<0.001
LEACH	HND	-83.7%	<0.001
LEACH	LND	-56.9%	<0.001

Table 14. Real map (80 OpenStreetMap buildings). All protocols except LEACH (mean ± CI of 30 runs) are deterministic: one run.

Protocol	FND	HND	LND	Hops/round
Direct	2	16	1097	1.0
LEACH	16 ± 2	66 ± 2	380 ± 35	1.4
PEGASIS	4	339	799	27.6
PEGASIS-2opt	160	493	975	32.8
PEGASIS-E	677	925	945	57.0
PEGASIS-MC	677	839	886	53.2

protocols shown, and the last-node winner is PEGASIS-2opt in five and PEGASIS-E in four combinations. Lifetimes fall as retries and sleep energy increase, as expected, but the ranking of FND and HND is stable. Because E_0 is normalised these are *relative* statements; with realistic battery capacities and sleep currents the idle energy would dominate for hourly reporting and routing would matter far less, which is a limitation of any radio-only model. A few cells are not monotonic (for instance PEGASIS HND rises from 157 to 167 when sleep energy is added at 2.0 attempts); we did not investigate this and treat small differences as noise from the discrete dynamics of chain re-building.

12. ON SIONNA AND 3D BUILDING MODELS

Sionna [14] is NVIDIA's open-source library for physical-layer research; its ray-tracing module (Sionna RT) computes radio propagation in 3D scenes. According to the documentation and tutorials we read, scenes are loaded from Mitsuba 3 XML files, which are typically produced in Blender with the Mitsuba add-on, and OpenStreetMap buildings can be imported to Blender with an add-on; two issues in the Sionna repository report problems on Windows. Consequently, files from Revit, SketchUp (`.skp`), STEP (`.stp`), OBJ or STL are *not* read directly as far as we found; a plausible route is to import an OBJ/STL mesh into Blender and export it as a Mitsuba scene, while `.skp` and `.stp` would first be converted to a mesh format. **We did not test this route.** Using Sionna RT would replace the d^2/d^4 path-loss model by ray-traced path loss between the node positions, which would capture shadowing by real buildings, but it belongs to the physical layer, requires a heavier set-up, and the protocol ranking would then have to be re-checked. We regard it as future work, not as part of the results of this report.

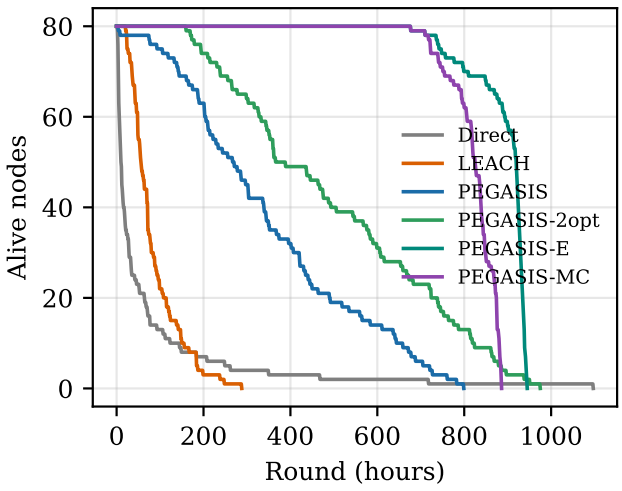


Figure 11. Alive nodes vs. round on the real map.

13. LIMITATIONS AND THREATS TO VALIDITY

- **Normalised energy.** $E_0 = 0.5$ J is the value conventional in the LEACH literature. Absolute lifetimes in days cannot be read from our results: real nodes also spend energy sleeping, sensing and listening, and ignoring this makes any "rounds → days" conversion meaningless. The results are *relative* comparisons only.
- **Synthetic deployment.** Positions are random or jittered-grid, not surveyed Nablus data. Step 1 of the tutorial explains how to replace them.
- **Idealised radio and MAC.** No collisions, interference, retransmissions, packet loss, antenna effects or rooftop obstacles; the d^2/d^4 model is a simplification.
- **Ideal fusion.** Fusing m packets always yields one packet of the same size.
- **Mid-round deaths.** A node that reaches zero energy during a round is still assumed to complete that round.
- **Global knowledge in PEGASIS.** The chain is built with full position knowledge; a distributed chain-building protocol would add set-up cost that we do not model.
- **Correlation, not causation** for the chain-link result.
- **Real map.** 80 buildings, one deterministic run per protocol (only LEACH is repeated); OpenStreetMap coverage may be incomplete; one building is assumed to hold one tank; the BS position is assumed.
- **Ablation scope.** Two geometries only (random 400 m blocks and one real map); the reason why cutting hurts is not isolated; the chosen objective (link energy) is a proxy for lifetime.
- **Realism.** Retries are modelled only as an energy factor (no packet loss or delivery ratio) and sleep energy as a constant; neither is calibrated to real hardware.
- **Novelty.** The ideas are variations of published ones and no quantitative comparison with the published protocols was made (Section 10).
- **Repository review.** Only one repository README was read in full and no repository code was executed (Section 3).
- **Motivation.** The statistics in the introduction are quoted from the cited reports and were not independently verified; they describe the West Bank as a whole, not Nablus alone.

14. CONCLUSION AND FUTURE WORK

We turned a small local problem, the lifetime of a tank-level sensor network in a Nablus residential block, into a fully reproducible study of energy-aware routing. The GitHub `leach` topic contains no PEGASIS implementation, so we built a documented, NumPy-only simulator on the same radio model as the best-documented Python repository and verified it with hand-computed tests. Over 50 paired deployments PEGASIS outlived LEACH by 47.6% (FND), 95.1% (HND) and 34.6% (LND), and Direct transmission lost its first node after about 22 rounds; the price of PEGASIS is a latency of about 63 sequential transmissions per round, and its FND is dominated by the longest chain link ($\rho = -0.99$). Two further ideas, a 2-opt chain and an energy-aware leader, were evaluated with a full ablation and on 80 real buildings from OpenStreetMap: together (PEGASIS-E) they raise first-node death from 38 ± 13 to 569 ± 43 rounds on random layouts and from 4 to 677 rounds on the real map, at roughly twice the latency, whereas cutting the chain at long links turned out to hurt. The whole study is packaged as a tested platform with API, database, dashboard and Node-RED integration.

Future work: (i) replace the synthetic layout by surveyed tank coordinates; (ii) a quantitative comparison with the published protocols EEPB, IEEPB and EECB; (iii) add LEACH-C, HEED and multi-hop variants; (iv) model packet loss and delivery ratio explicitly and calibrate sleep energy on hardware; (v) validate on a small hardware testbed (for instance LoRa or ESP32 nodes with ultrasonic level sensors) to calibrate the radio model on real rooftops.

ACKNOWLEDGMENT

The authors thank Dr. Jehad M. Hamamreh, founder of the WISLAB research group (`https://wislabi.com`), for supervising this work and for the tutorial-style paper [6] that served as a format model.

DATA AND CODE AVAILABILITY

All code, per-seed raw results (`results/baseline_raw.csv`) and the machine-readable summary (`results/summary.json`) are included with this report; see Appendix A for the complete source.

REFERENCES

[1] W. R. Heinzelman, A. Chandrakasan, and H. Balakrishnan, "Energy-efficient communication protocol for wireless microsensor networks," in *Proc. 33rd Hawaii Int. Conf. System Sciences (HICSS)*, Maui, HI, USA, Jan. 2000.

[2] W. B. Heinzelman, A. P. Chandrakasan, and H. Balakrishnan, "An application-specific protocol architecture for wireless microsensor networks," *IEEE Trans. Wireless Commun.*, vol. 1, no. 4, pp. 660–670, Oct. 2002.

[3] S. Lindsey and C. S. Raghavendra, "PEGASIS: Power-efficient gathering in sensor information systems," in *Proc. IEEE Aerospace Conf.*, vol. 3, 2002, pp. 1125–1130.

[4] I. F. Akyildiz, W. Su, Y. Sankarasubramaniam, and E. Cayirci, "Wireless sensor networks: A survey," *Computer Networks*, vol. 38, no. 4, pp. 393–422, Mar. 2002.

[5] J. N. Al-Karaki and A. E. Kamal, "Routing techniques in wireless sensor networks: A survey," *IEEE Wireless Commun.*, vol. 11, no. 6, pp. 6–28, Dec. 2004.

[6] S. Iqbal and J. M. Hamamreh, "A comprehensive tutorial on how to practically build and deploy 5G networks using open-source software and general-purpose, off-the-shelf hardware," *RS Open Journal on Innovative Communication Technologies*, vol. 2, Dec. 2021, DOI: 10.46470/03d8ffbd.4ccb7950.

[7] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[8] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[9] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[10] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[11] G. A. Croes, "A method for solving traveling-salesman problems," *Operations Research*, vol. 6, no. 6, pp. 791–812, 1958.

[12] OpenStreetMap contributors, "OpenStreetMap," data available under the Open Database Licence. [Online]. Available: `https://www.openstreetmap.org`

[13] OpenJS Foundation, "Node-RED: Low-code programming for event-driven applications." [Online]. Available: `https://nodered.org`

[14] NVIDIA, "Sionna: An open-source library for next-generation physical layer research." [Online]. Available: `https://github.com/NVlabs/sionna`

[15] B'Tselem, "Parched: Israel's policy of water deprivation in the West Bank," Apr. 2023. [Online]. Available: `https://www.btselem.org/sites/default/files/publications/202305_parched_eng.pdf`

[16] Palestinian Central Bureau of Statistics (PCBS) and Palestinian Water Authority (PWA), "Joint press release on the occasion of World Water Day," Ramallah, Palestine, Mar. 22, 2022. [Online]. Available: `https://www.pcbs.gov.ps/portals/_pcbs/PressRelease/Press_En_22-3-2022-Water-en.pdf`

[17] L. Yueyang, J. Hong, and Y. Guangxin, "An energy-efficient PEGASIS-based enhanced algorithm in wireless sensor networks," *China Communications*, 2006.

[18] F. Sen, Q. Bing, and T. Liangrui, "An improved energy-efficient PEGASIS-based protocol in wireless sensor networks," in *Proc. 8th Int. Conf. Fuzzy Systems and Knowledge Discovery (FSKD)*, vol. 4, 2011, pp. 2230–2233.

[19] M. R. Jafri, N. Javaid, A. Javaid, and Z. A. Khan, "Maximizing the lifetime of multi-chain PEGASIS using sink mobility," arXiv:1303.4347. [Online]. Available: `https://arxiv.org/pdf/1303.4347`

[20] Aarti and S. K. Tomar, "A parametric chain based routing approach for underwater sensor network," *Int. J. Computer Trends and Technology*, vol. 4, no. 5, May 2013, arXiv:1402.5748.

[21] B. A. Ahmmad and S. A. Alabady, "Energy-efficient routing protocol developed for internet of things networks," *IET Quantum Communication*, vol. 4, no. 1, pp. 25–38, Mar. 2023, doi: 10.1049/qtc2.12051.

[22] GitHub topic “leach” and repository `Nachi28/WSN_LEACH`. [Online]. Available: `https://github.com/topics/leach`, `https://github.com/Nachi28/WSN_LEACH`. Accessed: Oct. 10, 2026.

A. COMPLETE SOURCE CODE

A.1 `code/wsn_sim.py` – simulator and self-tests

```
"""
wsn_sim.py -- Energy-aware WSN routing simulator: LEACH vs PEGASIS (+ Direct baseline)
================================================================================
Author : Ibrahim Abuzant (An-Najah National University)
Supervisor : Prof. Jihad Hamamreh

What this file contains
-----------------------
1. Params          : every physical/protocol constant in ONE place (first-order radio model).
2. e_tx / e_rx     : radio energy equations (Heinzelman et al.).
3. make_layout     : node deployment (random or jittered street grid).
4. run_direct      : baseline, every node transmits straight to the sink.
5. run_leach       : LEACH (randomised cluster heads, threshold T(n)).
6. run_pegasis     : PEGASIS (greedy chain, rotating leader, data fusion along the chain).
7. selftest        : hand-computed checks that prove the simulator is correct.

Only NumPy is required.  Run `python wsn_sim.py` to execute the self-tests.
"""
from __future__ import annotations
from dataclasses import dataclass, replace
import numpy as np


# ---------------------------------------------------------------------------- #
# 1. PARAMETERS
# ---------------------------------------------------------------------------- #
@dataclass(frozen=True)
class Params:
    # ---- deployment (neighbourhood scale) ----
    n_nodes: int = 100            # one node per building (rooftop water tank)
    field: float = 200.0          # side of the square neighbourhood block [m]
    sink: tuple = (100.0, 250.0)  # base station / community point [m]
    # ---- energy model (Heinzelman et al., 2000/2002) ----
    e0: float = 0.5               # initial energy per node [J]  (normalised, see report)
    e_elec: float = 50e-9         # electronics energy  [J/bit]
    eps_fs: float = 10e-12        # free-space amplifier [J/bit/m^2]
    eps_mp: float = 0.0013e-12    # multipath amplifier  [J/bit/m^4]
    e_da: float = 5e-9            # data-fusion energy   [J/bit/signal]
    # ---- packets ----
    k_data: int = 4000            # data packet [bits]
    k_ctrl: int = 200             # control packet (ADV / JOIN / SCHEDULE) [bits]
    # ---- LEACH ----
    p: float = 0.05               # desired fraction of cluster heads per round
    # ---- optional realism (defaults = off, so earlier results are unchanged) ----
    attempts: float = 1.0         # mean transmissions per packet (ARQ retries): 1.5 = 50 % extra tx AND rx energy
    e_sleep: float = 0.0          # idle / sleep energy per living node per round [J]
    # ---- simulation ----
    max_rounds: int = 30000

    @property
    def d0(self) -> float:
        """Crossover distance between free-space and multipath models [m]."""
        return float(np.sqrt(self.eps_fs / self.eps_mp))


# ---------------------------------------------------------------------------- #
# 2. RADIO MODEL
# ---------------------------------------------------------------------------- #
def e_tx(P: Params, k, d):
    """Energy to transmit k bits over distance d (vectorised over d)."""
    d = np.asarray(d, dtype=float)
    amp = np.where(d < P.d0, P.eps_fs * d**2, P.eps_mp * d**4)
    return (k * P.e_elec + k * amp) * P.attempts


def e_rx(P: Params, k):
    """Energy to receive k bits."""
    return k * P.e_elec * P.attempts


def e_fuse(P: Params, n_signals, k=None):
    """Energy to fuse n_signals packets of k bits into one (0 if nothing to fuse)."""
    k = P.k_data if k is None else k
    n_signals = np.asarray(n_signals)
    return np.where(n_signals >= 2, n_signals * k * P.e_da, 0.0)


# ---------------------------------------------------------------------------- #
# 3. DEPLOYMENT
# ---------------------------------------------------------------------------- #
def make_layout(P: Params, rng: np.random.Generator, kind: str = "random") -> np.ndarray:
    """Return an (N,2) array of node coordinates.
    kind='random' : uniform random in the block (standard in the LEACH literature)
    kind='grid'   : jittered street grid, a crude model of buildings along streets
    """
    N, L = P.n_nodes, P.field
    if kind == "random":
```

```
        return rng.uniform(0, L, size=(N, 2))
    if kind == "grid":
        side = int(np.ceil(np.sqrt(N)))
        step = L / side
        xs, ys = np.meshgrid((np.arange(side) + 0.5) * step, (np.arange(side) + 0.5) * step)
        pts = np.column_stack([xs.ravel(), ys.ravel()])
        pts = pts[rng.permutation(len(pts))[:N]]
        pts = pts + rng.normal(0, 0.15 * step, size=pts.shape)
        return np.clip(pts, 0, L)
    raise ValueError(kind)


def _dist_to_sink(P, pos):
    return np.hypot(pos[:, 0] - P.sink[0], pos[:, 1] - P.sink[1])


# ---------------------------------------------------------------------------- #
# helpers shared by all protocols
# ---------------------------------------------------------------------------- #
class _Tracker:
    """Records alive-count / residual energy per round and the key lifetime metrics."""

    def __init__(self, P, record=False):
        self.P = P
        self.record = record        # keep a full per-round log (used by the live dashboard)
        self.log = []
        self.role = None            # who did what this round (set by the protocol before end_round)
        self.alive_curve, self.energy_curve = [], []
        self.fnd = self.hnd = self.lnd = None
        self.death_round = np.full(P.n_nodes, np.nan)
        self.sink_packets = 0       # packets that reached the sink
        self.readings = 0           # sensor readings represented in those packets
        self.hops = []              # sequential transmissions needed per round (latency proxy)

    def end_round(self, r, E, alive):
        P = self.P
        if P.e_sleep:
            E[alive] -= P.e_sleep               # idle / sleep consumption of every living node
        died = alive & (E <= 0)
        E[died] = 0.0
        alive[died] = False
        self.death_round[died & np.isnan(self.death_round)] = r + 1
        n_alive = int(alive.sum())
        self.alive_curve.append(n_alive)
        self.energy_curve.append(float(E.sum()))
        if self.record:
            self.log.append(dict(E=E.astype(np.float32), alive=alive.copy(), role=self.role))
        self.role = None
        if self.fnd is None and n_alive < P.n_nodes:
            self.fnd = r + 1
        if self.hnd is None and n_alive <= P.n_nodes / 2:
            self.hnd = r + 1
        if self.lnd is None and n_alive == 0:
            self.lnd = r + 1
        return n_alive == 0

    def result(self, E, extra=None):
        out = dict(
            alive=np.array(self.alive_curve), energy=np.array(self.energy_curve),
            fnd=self.fnd, hnd=self.hnd, lnd=self.lnd,
            death_round=self.death_round, sink_packets=self.sink_packets,
            readings=self.readings, hops=float(np.mean(self.hops)) if self.hops else 0.0,
            E_final=E.copy(),
        )
        if self.record:
            out["log"] = self.log
        if extra:
            out.update(extra)
        return out


# ---------------------------------------------------------------------------- #
# 4. DIRECT TRANSMISSION (baseline)
# ---------------------------------------------------------------------------- #
def run_direct(P: Params, pos: np.ndarray, rng=None, snap_round=None, record=False):
    N = P.n_nodes
    E = np.full(N, P.e0)
    alive = np.ones(N, bool)
    ds = _dist_to_sink(P, pos)
    T = _Tracker(P, record)
    cost = e_tx(P, P.k_data, ds)
    for r in range(P.max_rounds):
        idx = np.flatnonzero(alive)
        E[idx] -= cost[idx]
        T.sink_packets += idx.size
        T.readings += idx.size
        T.hops.append(1)
        T.role = dict(kind="direct")
        if T.end_round(r, E, alive):
            break
    return T.result(E)
```

```python


# ------------------------------------------------------------------------ #
# 5. LEACH
# ------------------------------------------------------------------------ #
def run_leach(P: Params, pos: np.ndarray, rng: np.random.Generator, snap_round: int | None = None, record=False):
    """LEACH with set-up phase (ADV, JOIN, SCHEDULE) and steady-state phase."""
    N = P.n_nodes
    E = np.full(N, P.e0)
    alive = np.ones(N, bool)
    eligible = np.ones(N, bool)             # "has not been a CH in the current epoch"
    cycle = int(round(1 / P.p))             # epoch length = 1/p rounds
    ds = _dist_to_sink(P, pos)
    T = _Tracker(P, record)
    snap = None

    for r in range(P.max_rounds):
        if r % cycle == 0:
            eligible[:] = alive             # new epoch: every living node is eligible again
        idx = np.flatnonzero(alive)
        # threshold T(n) = p / (1 - p * (r mod 1/p))
        thr = P.p / (1 - P.p * (r % cycle))
        is_ch = eligible & alive & (rng.random(N) < thr)
        heads = np.flatnonzero(is_ch)
        eligible[heads] = False

        if heads.size == 0:
            # rare: nobody volunteered -> fall back to direct transmission this round
            E[idx] -= e_tx(P, P.k_data, ds[idx])
            T.sink_packets += idx.size
            T.readings += idx.size
            T.hops.append(1)
            T.role = dict(kind="direct")
        else:
            members = idx[~is_ch[idx]]
            # ---- set-up phase ----
            # (a) every CH broadcasts an ADV strong enough to reach the farthest living node
            for h in heads:
                dfar = np.max(np.hypot(pos[idx, 0] - pos[h, 0], pos[idx, 1] - pos[h, 1]))
                E[h] -= e_tx(P, P.k_ctrl, dfar)
            # (b) every member hears all ADVs, joins the nearest CH
            E[members] -= e_rx(P, P.k_ctrl) * heads.size
            if members.size:
                dm = np.hypot(pos[members, None, 0] - pos[None, heads, 0],
                              pos[members, None, 1] - pos[None, heads, 1])
                nearest = dm.argmin(axis=1)
                dmin = dm[np.arange(members.size), nearest]
                E[members] -= e_tx(P, P.k_ctrl, dmin)                 # JOIN message
                n_mem = np.bincount(nearest, minlength=heads.size)    # members per CH
                E[heads] -= e_rx(P, P.k_ctrl) * n_mem                 # CH hears JOINs
                # (c) CH broadcasts TDMA schedule to its members
                dmax = np.zeros(heads.size)
                np.maximum.at(dmax, nearest, dmin)
                has = n_mem > 0
                E[heads[has]] -= e_tx(P, P.k_ctrl, dmax[has])
                E[members] -= e_rx(P, P.k_ctrl)
                # ---- steady-state phase ----
                E[members] -= e_tx(P, P.k_data, dmin)                 # member -> CH
                E[heads] -= e_rx(P, P.k_data) * n_mem                 # CH receives
                E[heads] -= e_fuse(P, n_mem + 1)                      # CH fuses own + members
            else:
                nearest = np.array([], int)
            E[heads] -= e_tx(P, P.k_data, ds[heads])                  # CH -> sink
            T.sink_packets += heads.size
            T.readings += idx.size
            T.hops.append(2 if members.size else 1)
            T.role = dict(kind="leach", heads=heads.copy(), members=members.copy(),
                          assign=heads[nearest].copy() if members.size else np.array([], int))
            if snap_round is not None and r == snap_round:
                snap = dict(heads=heads.copy(), members=members.copy(),
                            assign=heads[nearest].copy() if members.size else np.array([], int))
        if T.end_round(r, E, alive):
            break
    return T.result(E, dict(snapshot=snap))


# ------------------------------------------------------------------------ #
# 6. PEGASIS
# ------------------------------------------------------------------------ #
def build_chain(pos: np.ndarray, idx: np.ndarray, ds: np.ndarray) -> np.ndarray:
    """Greedy chain: start at the node farthest from the sink, then repeatedly
    append the closest node that is not yet on the chain."""
    remaining = list(idx)
    start = max(remaining, key=lambda i: ds[i])
    chain = [start]
    remaining.remove(start)
    rem = np.array(remaining, dtype=int)
    while rem.size:
        last = chain[-1]
        d = np.hypot(pos[rem, 0] - pos[last, 0], pos[rem, 1] - pos[last, 1])
        j = int(d.argmin())
```

```
        chain.append(int(rem[j]))
        rem = np.delete(rem, j)
    return np.array(chain, dtype=int)


def _gather_cost(P, seg, L, n, d_bs):
    """Energy spent by each of the n nodes of ONE chain in one round (leader at chain position L).
    seg[i] = length of the link between chain positions i and i+1;  d_bs = leader-to-base-station distance."""
    k = P.k_data
    cost = np.zeros(n)
    if L > 0:                                          # left side passes data towards the leader
        cost[0:L] += e_tx(P, k, seg[0:L])
        cost[1:L + 1] += e_rx(P, k)
        if L > 1:
            cost[1:L] += e_fuse(P, 2)
    if L < n - 1:                                      # right side passes data towards the leader
        cost[L + 1:n] += e_tx(P, k, seg[L:n - 1])
        cost[L:n - 1] += e_rx(P, k)
        if L < n - 2:
            cost[L + 1:n - 1] += e_fuse(P, 2)
    incoming = int(L > 0) + int(L < n - 1)             # leader fuses everything, sends ONE packet to the BS
    cost[L] += e_fuse(P, incoming + 1)
    cost[L] += e_tx(P, k, d_bs)
    return cost


def _greedy_chain(P, pos, idx, ds):
    return build_chain(pos, idx, ds)


def build_chain_2opt(P, pos, idx, ds, max_passes=30):
    """Chain = open Hamiltonian path.  Start from the greedy chain, then apply the classic 2-opt
    local search (reverse a segment whenever that lowers the total link ENERGY) until no move helps."""
    chain = build_chain(pos, idx, ds)
    n = len(chain)
    if n < 4:
        return chain
    x, y = pos[chain, 0], pos[chain, 1]
    C = e_tx(P, P.k_data, np.hypot(x[:, None] - x[None, :], y[:, None] - y[None, :]))   # link-energy matrix
    order = np.arange(n)
    for _ in range(max_passes):
        improved = False
        for i in range(n - 1):
            a, b = (order[i - 1] if i > 0 else -1), order[i]
            js = np.arange(i + 1, n)
            c = order[js]
            dn = np.where(js < n - 1, order[np.minimum(js + 1, n - 1)], -1)
            old = (C[a, b] if a >= 0 else 0.0) + np.where(dn >= 0, C[c, dn], 0.0)
            new = (C[a, c] if a >= 0 else 0.0) + np.where(dn >= 0, C[b, dn], 0.0)
            gain = old - new
            k = int(gain.argmax())
            if gain[k] > 1e-15:
                j = int(js[k])
                order[i:j + 1] = order[i:j + 1][::-1].copy()
                improved = True
        if not improved:
            break
    return chain[order]


def run_pegasis(P: Params, pos: np.ndarray, rng=None, snap_round: int | None = None, record=False,
                chain_fn=_greedy_chain):
    N = P.n_nodes
    E = np.full(N, P.e0)
    alive = np.ones(N, bool)
    ds = _dist_to_sink(P, pos)
    T = _Tracker(P, record)
    chain, last_alive_key, snap = None, None, None
    k = P.k_data

    for r in range(P.max_rounds):
        idx = np.flatnonzero(alive)
        n = idx.size
        key = idx.tobytes()
        if key != last_alive_key:                      # rebuild chain only when a node died
            chain = chain_fn(P, pos, idx, ds)
            seg = np.hypot(np.diff(pos[chain, 0]), np.diff(pos[chain, 1]))   # link lengths
            last_alive_key = key
        leader_node = idx[r % n]                       # rotating leader: i mod N
        L = int(np.flatnonzero(chain == leader_node)[0])
        cost = _gather_cost(P, seg, L, n, ds[chain[L]])

        E[chain] -= cost
        T.sink_packets += 1
        T.readings += n
        T.hops.append(max(L, n - 1 - L) + 1)           # sequential transmissions (latency proxy)
        T.role = dict(kind="pegasis", chain=chain, leader=int(leader_node))
        if snap_round is not None and r == snap_round:
            snap = dict(chain=chain.copy(), leader=int(leader_node))
        if T.end_round(r, E, alive):
            break
```

```
    return T.result(E, dict(snapshot=snap))


def run_pegasis_2opt(P, pos, rng=None, snap_round=None, record=False):
    """PEGASIS whose chain is improved with 2-opt local search (a TSP heuristic) instead of the greedy chain."""
    return run_pegasis(P, pos, rng, snap_round, record, chain_fn=build_chain_2opt)


def run_pegasis_variant(P, pos, rng=None, snap_round=None, record=False,
                        chain="2opt", cut=True, leader="energy", threshold=None):
    """One runner for the whole PEGASIS family; every option is an independent switch (used for the ablation).
       chain  : "greedy" (original PEGASIS) | "2opt" (greedy + 2-opt local search)
       cut    : False = one chain | True = cut the chain at every link >= threshold (default d0)
       leader : "rotate" = round-robin inside each chain (original PEGASIS rule)
                "energy" = node with the largest residual_energy / E_tx(to BS)
       (chain="greedy", cut=False, leader="rotate") is exactly the original PEGASIS."""
    N = P.n_nodes
    thr = P.d0 if threshold is None else threshold
    builder = build_chain_2opt if chain == "2opt" else _greedy_chain
    E = np.full(N, P.e0)
    alive = np.ones(N, bool)
    ds = _dist_to_sink(P, pos)
    T = _Tracker(P, record)
    last_key, segments = None, []
    for r in range(P.max_rounds):
        idx = np.flatnonzero(alive)
        key = idx.tobytes()
        if key != last_key:
            ch = builder(P, pos, idx, ds)
            links = np.hypot(np.diff(pos[ch, 0]), np.diff(pos[ch, 1]))
            parts = np.split(ch, np.flatnonzero(links >= thr) + 1) if cut else [ch]
            segments = [(p, np.hypot(np.diff(pos[p, 0]), np.diff(pos[p, 1]))) for p in parts]
            last_key = key
        leaders, chains, hops = [], [], 0
        for members, seg in segments:
            n_s = len(members)
            if leader == "energy":
                L = int((E[members] / e_tx(P, P.k_data, ds[members])).argmax())
            else:
                L = int(np.flatnonzero(members == np.sort(members)[r % n_s])[0])
            E[members] -= _gather_cost(P, seg, L, n_s, ds[members[L]])
            leaders.append(int(members[L])); chains.append(members)
            hops = max(hops, max(L, n_s - 1 - L) + 1)
        T.sink_packets += len(segments)
        T.readings += idx.size
        T.hops.append(hops)
        T.role = dict(kind="pegasis_mc", chains=chains, leaders=leaders)
        if T.end_round(r, E, alive):
            break
    return T.result(E)


def run_pegasis_mc(P, pos, rng=None, snap_round=None, record=False, threshold=None):
    """PEGASIS-MC (multi-chain): 2-opt chain, cut at links >= d0, energy-aware leader in every chain."""
    return run_pegasis_variant(P, pos, rng, snap_round, record, chain="2opt", cut=True, leader="energy",
                               threshold=threshold)


def run_pegasis_e(P, pos, rng=None, snap_round=None, record=False):
    """PEGASIS-E: 2-opt chain + energy-aware leader, ONE chain (the best variant found by the ablation study)."""
    return run_pegasis_variant(P, pos, rng, snap_round, record, chain="2opt", cut=False, leader="energy")


PROTOCOLS = {"Direct": run_direct, "LEACH": run_leach, "PEGASIS": run_pegasis, "PEGASIS-2opt": run_pegasis_2opt,
             "PEGASIS-E": run_pegasis_e, "PEGASIS-MC": run_pegasis_mc}


# ------------------------------------------------------------------------- #
# 7. SELF-TESTS  (hand-computed numbers; if these pass, the simulator is trustworthy)
# ------------------------------------------------------------------------- #
def selftest(verbose=True):
    P = Params()
    ok = True

    def check(name, cond):
        nonlocal ok
        ok &= bool(cond)
        if verbose:
            print(f"[{'PASS' if cond else 'FAIL'}] {name}")

    # T1: radio model is continuous at the crossover distance d0
    d0 = P.d0
    fs = P.k_data * P.eps_fs * d0**2
    mp = P.k_data * P.eps_mp * d0**4
    check("T1 free-space and multipath curves meet at d0 (%.2f m)" % d0, np.isclose(fs, mp))

    # T2: Direct transmission -> death round of every node equals ceil(E0/cost) analytically
    rng = np.random.default_rng(1)
    P2 = replace(P, n_nodes=20)
    pos = make_layout(P2, rng)
    res = run_direct(P2, pos)
```

```
    expected = np.ceil(P2.e0 / e_tx(P2, P2.k_data, _dist_to_sink(P2, pos)) - 1e-9)
    check("T2 Direct: simulated death rounds == analytic ceil(E0/E_tx)",
          np.all(np.abs(res["death_round"] - expected) <= 1) and res["fnd"] == res["death_round"].min())

    # T3: PEGASIS, 3 nodes on a line, ONE round, compare against independent hand formula
    P3 = replace(P, n_nodes=3, sink=(60.0, 100.0), max_rounds=1)
    pos3 = np.array([[0.0, 0.0], [30.0, 0.0], [60.0, 0.0]])
    res = run_pegasis(P3, pos3)
    k = P3.k_data
    tx = lambda d: k * P3.e_elec + k * P3.eps_fs * d**2           # free-space (d < d0)
    rx = k * P3.e_elec
    d_sink = np.hypot(60.0, 100.0)                                 # 116.6 m > d0 -> multipath model
    tx_sink = k * P3.e_elec + k * P3.eps_mp * d_sink**4
    exp = np.array([
        rx + 2 * k * P3.e_da + tx_sink,             # leader: rx from node1, fuse 2 signals, tx to sink
        rx + 2 * k * P3.e_da + tx(30.0),            # relay : rx from node2, fuse 2, tx to node0
        tx(30.0),                                   # end   : tx to node1
    ])
    check("T3 PEGASIS 3-node round matches hand calculation",
          np.allclose(P3.e0 - res["E_final"], exp, rtol=1e-9, atol=1e-15))

    # T4: energy never increases and alive count never increases (LEACH & PEGASIS)
    rng = np.random.default_rng(7)
    pos = make_layout(P, rng)
    for name, fn in (("LEACH", run_leach), ("PEGASIS", run_pegasis)):
        res = fn(P, pos, np.random.default_rng(3))
        mono = np.all(np.diff(res["energy"]) <= 1e-12) and np.all(np.diff(res["alive"]) <= 0)
        check(f"T4 {name}: residual energy and alive count are monotonically non-increasing", mono)

    # T5: ordering sanity -> farthest-from-sink node dies first under Direct
    res = run_direct(P, pos)
    check("T5 Direct: farthest node from the sink dies first",
          int(np.nanargmin(res["death_round"])) == int(np.argmax(_dist_to_sink(P, pos))))
    # T6: 2-opt chain is a permutation of the nodes and never costs more link-energy than the greedy chain
    rng = np.random.default_rng(11)
    pos6 = make_layout(P, rng)
    idx6 = np.arange(P.n_nodes)
    ds6 = _dist_to_sink(P, pos6)
    g, o = build_chain(pos6, idx6, ds6), build_chain_2opt(P, pos6, idx6, ds6)
    cost = lambda ch: float(e_tx(P, P.k_data, np.hypot(np.diff(pos6[ch, 0]), np.diff(pos6[ch, 1])))).sum())
    check("T6 2-opt chain: same node set, total link energy <= greedy (%.4f <= %.4f J)" % (cost(o), cost(g)),
          sorted(o.tolist()) == idx6.tolist() and cost(o) <= cost(g) + 1e-12)

    # T7: PEGASIS-MC, 3 nodes, one link longer than d0 -> 2 chains; compare one round with the hand calculation
    P7 = replace(P, n_nodes=3, sink=(0.0, 100.0), max_rounds=1)
    pos7 = np.array([[0.0, 0.0], [30.0, 0.0], [300.0, 0.0]])
    res = run_pegasis_mc(P7, pos7)
    k = P7.k_data
    txfs = lambda d: k * P7.e_elec + k * P7.eps_fs * d**2
    txmp = lambda d: k * P7.e_elec + k * P7.eps_mp * d**4
    rx = k * P7.e_elec
    exp7 = np.array([
        rx + 2 * k * P7.e_da + txmp(100.0),         # node0: leader of chain [1,0] (100 m > d0 -> multipath), rx, fuse 2, tx BS
        txfs(30.0),                                  # node1: end of chain, tx to node0
        txmp(np.hypot(300.0, 100.0)),                # node2: alone in its own chain, tx straight to BS
    ])
    check("T7 PEGASIS-MC: 2 chains, one round matches hand calculation",
          np.allclose(P7.e0 - res["E_final"], exp7, rtol=1e-9, atol=1e-15))
    # T8: the generic runner reproduces the ORIGINAL protocols exactly (same lifetimes, same final energies)
    pos8 = make_layout(P, np.random.default_rng(5))
    a, b = run_pegasis(P, pos8), run_pegasis_variant(P, pos8, chain="greedy", cut=False, leader="rotate")
    c, d = run_pegasis_2opt(P, pos8), run_pegasis_variant(P, pos8, chain="2opt", cut=False, leader="rotate")
    check("T8 generic runner == original PEGASIS and PEGASIS-2opt (identical lifetimes and energies)",
          (a["fnd"], a["hnd"], a["lnd"]) == (b["fnd"], b["hnd"], b["lnd"]) and np.allclose(a["energy"], b["energy"])
          and (c["fnd"], c["hnd"], c["lnd"]) == (d["fnd"], d["hnd"], d["lnd"]) and np.allclose(c["energy"], d["energy"]))

    # T9: realism switches -- retries cost energy, sleep costs energy, and defaults change nothing
    Pa, Ps = replace(P, attempts=2.0), replace(P, e_sleep=1e-3)
    base, dbl, slp = run_direct(P, pos8), run_direct(Pa, pos8), run_direct(Ps, pos8)
    check("T9 attempts=2 halves Direct lifetime (+-1 round) and sleep energy shortens it",
          abs(dbl["fnd"] * 2 - base["fnd"]) <= 2 and slp["fnd"] < base["fnd"] and replace(P).attempts == 1.0)
    return ok


if __name__ == "__main__":
    print("All self-tests passed." if selftest() else "SOME TESTS FAILED.")
```

A.2 `code/run_experiments.py` – experiments, tables and figures

```
"""
run_experiments.py -- reproduces EVERY number, table and figure in the report.

Usage:   python run_experiments.py            (about 1-2 minutes on a laptop)
Output:  ../results/*.tex  (LaTeX tables)   ../results/*.csv/json   ../figures/*.pdf|png

Nothing in the report is typed by hand: the LaTeX file \\input{}s the tables written here.
"""
from __future__ import annotations
import json, os, sys
```

```python
from dataclasses import replace
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats

from wsn_sim import (Params, PROTOCOLS, make_layout, run_leach, run_pegasis, selftest,
                     build_chain, _dist_to_sink)

HERE = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(HERE, "..", "figures")
RES = os.path.join(HERE, "..", "results")
os.makedirs(FIG, exist_ok=True)
os.makedirs(RES, exist_ok=True)

N_SEEDS_MAIN = 50
N_SEEDS_SENS = 30
COL = {"Direct": "#7f7f7f", "LEACH": "#d95f02", "PEGASIS": "#1b6ca8"}
MARK = {"Direct": "s", "LEACH": "o", "PEGASIS": "^"}

plt.rcParams.update({
    "font.family": "serif", "font.size": 9, "axes.labelsize": 9, "axes.titlesize": 9,
    "legend.fontsize": 8, "xtick.labelsize": 8, "ytick.labelsize": 8,
    "axes.grid": True, "grid.alpha": 0.3, "figure.dpi": 150, "savefig.bbox": "tight",
})


def savefig(fig, name):
    fig.savefig(os.path.join(FIG, name + ".pdf"))
    fig.savefig(os.path.join(FIG, name + ".png"), dpi=200)
    plt.close(fig)


# ---------------------------------------------------------------------- #
# core experiment routine
# ---------------------------------------------------------------------- #
def run_many(P: Params, seeds, layout="random", protocols=("Direct", "LEACH", "PEGASIS"), keep_curves=False):
    """Run each protocol on the SAME random deployment for every seed (paired design)."""
    out = {p: dict(fnd=[], hnd=[], lnd=[], hops=[], sink_packets=[], readings=[], alive=[], energy=[])
           for p in protocols}
    for s in seeds:
        pos = make_layout(P, np.random.default_rng(s), layout)
        for p in protocols:
            res = PROTOCOLS[p](P, pos, np.random.default_rng(10_000 + s))
            for key in ("fnd", "hnd", "lnd", "hops", "sink_packets", "readings"):
                out[p][key].append(res[key])
            if keep_curves:
                out[p]["alive"].append(res["alive"])
                out[p]["energy"].append(res["energy"])
    for p in protocols:
        for key in ("fnd", "hnd", "lnd", "hops", "sink_packets", "readings"):
            out[p][key] = np.array(out[p][key], dtype=float)
    return out


def ci95(x):
    x = np.asarray(x, float)
    if len(x) < 2:
        return 0.0
    return float(stats.t.ppf(0.975, len(x) - 1) * x.std(ddof=1) / np.sqrt(len(x)))


def ms(x, fmt="{:.0f}"):
    return (fmt + r" $\pm$ " + fmt).format(np.mean(x), ci95(x))


def pad_mean(curves, fill):
    L = max(len(c) for c in curves)
    M = np.full((len(curves), L), fill, float)
    for i, c in enumerate(curves):
        M[i, :len(c)] = c
        M[i, len(c):] = c[-1] if fill is None else fill
    return M


def write(name, text):
    with open(os.path.join(RES, name), "w") as f:
        f.write(text)


# ---------------------------------------------------------------------- #
def main():
    assert selftest(verbose=True), "self-tests failed -> stop"
    P = Params()
    summary = {"params": {k: (list(v) if isinstance(v, tuple) else v) for k, v in P.__dict__.items()},
               "d0": P.d0, "seeds_main": N_SEEDS_MAIN, "seeds_sens": N_SEEDS_SENS}

    # ================ E1: baseline ====================================== #
    print("E1 baseline ...")
    seeds = range(N_SEEDS_MAIN)
```

```
    base = run_many(P, seeds, keep_curves=True)

    # --- table 1: lifetime metrics
    rows = []
    for p in ("Direct", "LEACH", "PEGASIS"):
        d = base[p]
        rows.append(f"{p} & {ms(d['fnd'])} & {ms(d['hnd'])} & {ms(d['lnd'])} & "
                    f"{d['hops'].mean():.1f} & {np.mean(d['sink_packets'] / d['lnd']):.2f} & "
                    f"{ms(d['readings'] / 1e4, '{:.1f}')} \\\\")
    write("tab_baseline.tex",
          "\\begin{tabular}{lccccc}\n\\toprule\n"
          "Protocol & FND & HND & LND & Hops/round & Sink pkts/round & Readings ($\\times10^4$) \\\\\n"
          "\\midrule\n" + "\n".join(rows) + "\n\\bottomrule\n\\end{tabular}\n")

    # --- table 2: paired Wilcoxon tests LEACH vs PEGASIS
    wrows = []
    for key, lab in (("fnd", "FND"), ("hnd", "HND"), ("lnd", "LND")):
        a, b = base["LEACH"][key], base["PEGASIS"][key]
        w = stats.wilcoxon(a, b)
        better = "PEGASIS" if b.mean() > a.mean() else "LEACH"
        gain = (b.mean() - a.mean()) / a.mean() * 100
        wrows.append(f"{lab} & {a.mean():.0f} & {b.mean():.0f} & {gain:+.1f}\\% & "
                     f"{('$<10^{-3}$' if w.pvalue < 1e-3 else f'{w.pvalue:.3f}')} & {better} \\\\")
        summary[f"wilcoxon_{key}"] = dict(p=float(w.pvalue), leach=float(a.mean()), pegasis=float(b.mean()))
    write("tab_wilcoxon.tex",
          "\\begin{tabular}{lccccc}\n\\toprule\n"
          "Metric & LEACH mean & PEGASIS mean & PEGASIS vs LEACH & $p$ (Wilcoxon) & Longer-lived \\\\\n"
          "\\midrule\n" + "\n".join(wrows) + "\n\\bottomrule\n\\end{tabular}\n")

    # --- summary json
    summary["baseline"] = {p: {k: float(np.mean(v)) for k, v in base[p].items() if k not in ("alive", "energy")}
                           for p in base}
    summary["baseline_ci"] = {p: {k: ci95(v) for k, v in base[p].items() if k in ("fnd", "hnd", "lnd")}
                              for p in base}

    # --- csv of raw per-seed values (for the appendix / reproducibility)
    with open(os.path.join(RES, "baseline_raw.csv"), "w") as f:
        f.write("seed,protocol,fnd,hnd,lnd,hops,sink_packets,readings\n")
        for i, s in enumerate(seeds):
            for p in base:
                d = base[p]
                f.write(f"{s},{p},{d['fnd'][i]:.0f},{d['hnd'][i]:.0f},{d['lnd'][i]:.0f},"
                        f"{d['hops'][i]:.3f},{d['sink_packets'][i]:.0f},{d['readings'][i]:.0f}\n")

    # --- Fig: alive nodes & residual energy vs round
    fig, ax = plt.subplots(figsize=(3.4, 2.5))
    for p in ("Direct", "LEACH", "PEGASIS"):
        M = pad_mean(base[p]["alive"], 0)
        m, sd = M.mean(0), M.std(0)
        x = np.arange(1, len(m) + 1)
        ax.plot(x, m, color=COL[p], label=p, lw=1.4)
        ax.fill_between(x, np.maximum(m - sd, 0), np.minimum(m + sd, P.n_nodes), color=COL[p], alpha=0.15, lw=0)
    ax.set_xlabel("Round"); ax.set_ylabel("Alive nodes"); ax.legend(frameon=False)
    savefig(fig, "fig_alive")

    fig, ax = plt.subplots(figsize=(3.4, 2.5))
    for p in ("Direct", "LEACH", "PEGASIS"):
        M = pad_mean(base[p]["energy"], 0)
        m = M.mean(0)
        ax.plot(np.arange(1, len(m) + 1), m, color=COL[p], label=p, lw=1.4)
    ax.set_xlabel("Round"); ax.set_ylabel("Total residual energy (J)"); ax.legend(frameon=False)
    savefig(fig, "fig_energy")

    # --- Fig: lifetime bars
    fig, ax = plt.subplots(figsize=(3.4, 2.5))
    w = 0.26
    for j, p in enumerate(("Direct", "LEACH", "PEGASIS")):
        vals = [base[p][k].mean() for k in ("fnd", "hnd", "lnd")]
        errs = [ci95(base[p][k]) for k in ("fnd", "hnd", "lnd")]
        ax.bar(np.arange(3) + (j - 1) * w, vals, w, yerr=errs, capsize=2, color=COL[p], label=p)
    ax.set_xticks(range(3)); ax.set_xticklabels(["FND", "HND", "LND"])
    ax.set_ylabel("Rounds (mean, 95% CI)"); ax.legend(frameon=False)
    savefig(fig, "fig_lifetime")

    # --- Fig: death-round ECDF (how abruptly does each protocol lose nodes?)
    fig, ax = plt.subplots(figsize=(3.4, 2.5))
    for p in ("Direct", "LEACH", "PEGASIS"):
        M = pad_mean(base[p]["alive"], 0)
        dead_frac = 1 - M.mean(0) / P.n_nodes
        ax.plot(np.arange(1, len(dead_frac) + 1), dead_frac * 100, color=COL[p], label=p, lw=1.4)
    ax.set_xlabel("Round"); ax.set_ylabel("Dead nodes (%)"); ax.legend(frameon=False)
    savefig(fig, "fig_dead_pct")

    # --- Fig: snapshot of one LEACH round and one PEGASIS round on the same deployment
    pos = make_layout(P, np.random.default_rng(0))
    rl = run_leach(P, pos, np.random.default_rng(10_000), snap_round=50)
    rp = run_pegasis(P, pos, None, snap_round=50)
    fig, axs = plt.subplots(1, 2, figsize=(6.9, 3.4), sharey=True)
    for ax, title in zip(axs, ("(a) LEACH, round 51", "(b) PEGASIS, round 51")):
        ax.add_patch(plt.Rectangle((0, 0), P.field, P.field, fill=False, ec="k", lw=0.8, ls="--"))
        ax.set_xlim(-10, P.field + 10); ax.set_ylim(-10, P.sink[1] + 20)
```

```
        ax.set_aspect("equal"); ax.set_title(title); ax.set_xlabel("x (m)")
        ax.plot(*P.sink, marker="*", ms=14, color="gold", mec="k", ls="none", zorder=5, label="Sink")
    axs[0].set_ylabel("y (m)")
    s = rl["snapshot"]
    cmap = plt.get_cmap("tab10")
    for ci, h in enumerate(s["heads"]):
        mem = s["members"][s["assign"] == h]
        for m in mem:
            axs[0].plot([pos[m, 0], pos[h, 0]], [pos[m, 1], pos[h, 1]], color=cmap(ci % 10), lw=0.5, alpha=0.6)
        axs[0].scatter(pos[mem, 0], pos[mem, 1], s=9, color=cmap(ci % 10))
        axs[0].plot([pos[h, 0], P.sink[0]], [pos[h, 1], P.sink[1]], color=cmap(ci % 10), lw=0.9, ls=":")
        axs[0].scatter(pos[h, 0], pos[h, 1], s=55, marker="s", color=cmap(ci % 10), ec="k", zorder=4)
    s = rp["snapshot"]; ch = s["chain"]
    axs[1].plot(pos[ch, 0], pos[ch, 1], color=COL["PEGASIS"], lw=0.7, marker="o", ms=2.5)
    ld = s["leader"]
    axs[1].plot([pos[ld, 0], P.sink[0]], [pos[ld, 1], P.sink[1]], color="crimson", lw=1.1, ls=":")
    axs[1].scatter(pos[ld, 0], pos[ld, 1], s=70, marker="s", color="crimson", ec="k", zorder=4, label="Leader")
    axs[0].legend(loc="upper left", frameon=False); axs[1].legend(loc="upper left", frameon=False)
    savefig(fig, "fig_snapshot")

    # ================ E2: sink position ================================== #
    print("E2 sink position ...")
    sink_y = [100, 200, 250, 300, 400]
    seeds_s = range(N_SEEDS_SENS)
    e2 = {y: run_many(replace(P, sink=(100.0, float(y))), seeds_s) for y in sink_y}
    fig, axs = plt.subplots(1, 3, figsize=(6.9, 2.4), sharex=True)
    for ax, key, lab in zip(axs, ("fnd", "hnd", "lnd"), ("FND", "HND", "LND")):
        for p in ("Direct", "LEACH", "PEGASIS"):
            ax.errorbar(sink_y, [e2[y][p][key].mean() for y in sink_y],
                        yerr=[ci95(e2[y][p][key]) for y in sink_y], color=COL[p], marker=MARK[p],
                        ms=3.5, lw=1.1, capsize=2, label=p)
        ax.set_xlabel("Sink y-position (m)"); ax.set_title(lab)
    axs[0].set_ylabel("Rounds"); axs[0].legend(frameon=False)
    savefig(fig, "fig_sink")
    rows = []
    for y in sink_y:
        for p in ("LEACH", "PEGASIS"):
            pass
        rows.append(f"{y} & {e2[y]['LEACH']['fnd'].mean():.0f} & {e2[y]['PEGASIS']['fnd'].mean():.0f} & "
                    f"{e2[y]['LEACH']['hnd'].mean():.0f} & {e2[y]['PEGASIS']['hnd'].mean():.0f} & "
                    f"{e2[y]['LEACH']['lnd'].mean():.0f} & {e2[y]['PEGASIS']['lnd'].mean():.0f} \\\\")
    write("tab_sink.tex",
          "\\begin{tabular}{lccccc}\n\\toprule\n"
          " & \\multicolumn{2}{c}{FND} & \\multicolumn{2}{c}{HND} & \\multicolumn{2}{c}{LND} \\\\\n"
          "\\cmidrule(lr){2-3}\\cmidrule(lr){4-5}\\cmidrule(lr){6-7}\n"
          "Sink $y$ (m) & LEACH & PEGASIS & LEACH & PEGASIS & LEACH & PEGASIS \\\\\n\\midrule\n"
          + "\n".join(rows) + "\n\\bottomrule\n\\end{tabular}\n")

    # ================ E3: network size ================================== #
    print("E3 network size ...")
    Ns = [25, 50, 100, 150, 200]
    e3 = {n: run_many(replace(P, n_nodes=n), seeds_s, protocols=("LEACH", "PEGASIS")) for n in Ns}
    fig, axs = plt.subplots(1, 3, figsize=(6.9, 2.4), sharex=True)
    for ax, key, lab in zip(axs, ("fnd", "hnd", "lnd"), ("FND", "HND", "LND")):
        for p in ("LEACH", "PEGASIS"):
            ax.errorbar(Ns, [e3[n][p][key].mean() for n in Ns], yerr=[ci95(e3[n][p][key]) for n in Ns],
                        color=COL[p], marker=MARK[p], ms=3.5, lw=1.1, capsize=2, label=p)
        ax.set_xlabel("Number of nodes $N$"); ax.set_title(lab)
    axs[0].set_ylabel("Rounds"); axs[0].legend(frameon=False)
    savefig(fig, "fig_nodes")
    rows = [f"{n} & {e3[n]['LEACH']['fnd'].mean():.0f} & {e3[n]['PEGASIS']['fnd'].mean():.0f} & "
            f"{e3[n]['LEACH']['hnd'].mean():.0f} & {e3[n]['PEGASIS']['hnd'].mean():.0f} & "
            f"{e3[n]['LEACH']['lnd'].mean():.0f} & {e3[n]['PEGASIS']['lnd'].mean():.0f} & "
            f"{e3[n]['PEGASIS']['hops'].mean():.1f} \\\\" for n in Ns]
    write("tab_nodes.tex",
          "\\begin{tabular}{lccccccc}\n\\toprule\n"
          " & \\multicolumn{2}{c}{FND} & \\multicolumn{2}{c}{HND} & \\multicolumn{2}{c}{LND} & PEGASIS \\\\\n"
          "\\cmidrule(lr){2-3}\\cmidrule(lr){4-5}\\cmidrule(lr){6-7}\n"
          "$N$ & LEACH & PEGASIS & LEACH & PEGASIS & LEACH & PEGASIS & hops/round \\\\\n\\midrule\n"
          + "\n".join(rows) + "\n\\bottomrule\n\\end{tabular}\n")

    # ================ E4: LEACH p ================================== #
    print("E4 LEACH p ...")
    ps = [0.02, 0.05, 0.10, 0.15, 0.20]
    e4 = {pp: run_many(replace(P, p=pp), seeds_s, protocols=("LEACH",)) for pp in ps}
    peg_ref = run_many(P, seeds_s, protocols=("PEGASIS",))["PEGASIS"]
    fig, axs = plt.subplots(1, 3, figsize=(6.9, 2.4), sharex=True)
    for ax, key, lab in zip(axs, ("fnd", "hnd", "lnd"), ("FND", "HND", "LND")):
        ax.errorbar(ps, [e4[pp]["LEACH"][key].mean() for pp in ps], yerr=[ci95(e4[pp]["LEACH"][key]) for pp in ps],
                    color=COL["LEACH"], marker="o", ms=3.5, lw=1.1, capsize=2, label="LEACH")
        ax.axhline(peg_ref[key].mean(), color=COL["PEGASIS"], ls="--", lw=1.1, label="PEGASIS (reference)")
        ax.set_xlabel("LEACH $p$"); ax.set_title(lab)
    axs[0].set_ylabel("Rounds"); axs[0].legend(frameon=False)
    savefig(fig, "fig_p")
    rows = [f"{pp:.2f} & {ms(e4[pp]['LEACH']['fnd'])} & {ms(e4[pp]['LEACH']['hnd'])} & {ms(e4[pp]['LEACH']['lnd'])} \\\\"
            for pp in ps]
    write("tab_p.tex",
          "\\begin{tabular}{lccc}\n\\toprule\n$p$ & FND & HND & LND \\\\\n\\midrule\n"
          + "\n".join(rows) + "\n\\bottomrule\n\\end{tabular}\n")

    # ================ E5: layout ================================== #
```

```
    print("E5 layout ...")
    e5 = {lay: run_many(P, seeds_s, layout=lay, protocols=("LEACH", "PEGASIS")) for lay in ("random", "grid")}
    rows = []
    for lay, nm in (("random", "Uniform random"), ("grid", "Jittered street grid")):
        for p in ("LEACH", "PEGASIS"):
            d = e5[lay][p]
            rows.append(f"{nm} & {p} & {ms(d['fnd'])} & {ms(d['hnd'])} & {ms(d['lnd'])} \\\\")
    write("tab_layout.tex",
          "\\begin{tabular}{llccc}\n\\toprule\nLayout & Protocol & FND & HND & LND \\\\\n\\midrule\n"
          + "\n".join(rows) + "\n\\bottomrule\n\\end{tabular}\n")

    # ================ E6: why is PEGASIS FND so variable? ================ #
    print("E6 chain-link analysis ...")
    maxlink, fnd_p, nlong = [], [], []
    for i, sd in enumerate(range(N_SEEDS_MAIN)):
        pos_i = make_layout(P, np.random.default_rng(sd))
        ds_i = _dist_to_sink(P, pos_i)
        ch = build_chain(pos_i, np.arange(P.n_nodes), ds_i)
        seg = np.hypot(np.diff(pos_i[ch, 0]), np.diff(pos_i[ch, 1]))
        maxlink.append(seg.max())
        nlong.append(int((seg >= P.d0).sum()))
        fnd_p.append(base["PEGASIS"]["fnd"][i])
    maxlink, fnd_p, nlong = np.array(maxlink), np.array(fnd_p), np.array(nlong)
    rho, pval = stats.spearmanr(maxlink, fnd_p)
    summary["e6"] = dict(spearman_rho=float(rho), p=float(pval), mean_max_link=float(maxlink.mean()),
                         frac_layouts_with_link_ge_d0=float((nlong > 0).mean()),
                         mean_links_ge_d0=float(nlong.mean()), max_link_max=float(maxlink.max()))
    fig, ax = plt.subplots(figsize=(3.4, 2.5))
    ax.scatter(maxlink, fnd_p, s=14, color=COL["PEGASIS"], ec="k", lw=0.3)
    ax.axvline(P.d0, color="crimson", ls="--", lw=1, label=r"$d_0$ = %.1f m" % P.d0)
    ax.set_xlabel("Longest chain link (m)"); ax.set_ylabel("PEGASIS FND (rounds)")
    ax.legend(frameon=False)
    savefig(fig, "fig_chain_link")

    # ---- store everything the report text quotes
    summary["sink_sensitivity"] = {str(y): {p: {k: float(e2[y][p][k].mean()) for k in ("fnd", "hnd", "lnd")}
                                            for p in e2[y]} for y in sink_y}
    summary["node_sensitivity"] = {str(n): {p: {k: float(e3[n][p][k].mean()) for k in ("fnd", "hnd", "lnd", "hops")}
                                            for p in e3[n]} for n in Ns}
    summary["p_sensitivity"] = {str(pp): {k: float(e4[pp]["LEACH"][k].mean()) for k in ("fnd", "hnd", "lnd")}
                                for pp in ps}
    summary["layout"] = {lay: {p: {k: float(e5[lay][p][k].mean()) for k in ("fnd", "hnd", "lnd")}
                               for p in e5[lay]} for lay in e5}
    with open(os.path.join(RES, "summary.json"), "w") as f:
        json.dump(summary, f, indent=2)
    print("done.")


if __name__ == "__main__":
    main()
```

A.3 `code/ablation.py` *– ablation, realism study, parallel runner*

```
"""
ablation.py -- WHY does PEGASIS-MC work?  An ablation study + a realism sensitivity study, run in PARALLEL.

PEGASIS-MC changes three things at once.  Here every combination of the three switches is run:
    chain  : greedy | 2opt          (how the chain is built)
    cut    : no | yes               (cut the chain at links >= d0 = 87.7 m)
    leader : rotate | energy        (round-robin leader  vs  residual-energy / cost-to-BS leader)
-> 8 variants.  (greedy, no, rotate) is the original PEGASIS.

Usage:   python ablation.py                (30 random layouts + the real layout; uses all CPU cores)
         python ablation.py --workers 1    (sequential; gives identical numbers)
         python ablation.py --seeds 5      (quick test)
Needs numpy + scipy.  Writes ablation_results.md, ablation_table.tex, ablation_effects.tex,
realism_results.md, realism_table.tex next to this file.
"""
import argparse, itertools, json, os
from concurrent.futures import ProcessPoolExecutor
from dataclasses import replace
import numpy as np
from scipy import stats
from wsn_sim import Params, PROTOCOLS, make_layout, run_pegasis_variant

HERE = os.path.dirname(os.path.abspath(__file__))
REAL = os.path.join(HERE, "real_layout.json")
CHAINS, CUTS, LEADERS = ("greedy", "2opt"), (False, True), ("rotate", "energy")
VARIANTS = list(itertools.product(CHAINS, CUTS, LEADERS))
PR = replace(Params(), n_nodes=80, field=400.0, sink=(200.0, 450.0))          # random layouts: same size as the real block


def vname(v):
    return f"{v[0]}/{'cut' if v[1] else 'nocut'}/{v[2]}"


def setup(layout, seed):
    if layout == "real":
        L = json.load(open(REAL))
```

```
        pos = np.array(L["positions"])
        return replace(Params(), n_nodes=len(pos), field=float(L["field"]), sink=tuple(L["sink"])), pos
    return PR, make_layout(PR, np.random.default_rng(seed))


def job(args):                      # runs in a worker process; arguments and result are plain picklable data
    v, layout, seed = args
    P, pos = setup(layout, seed)
    r = run_pegasis_variant(P, pos, None, None, False, chain=v[0], cut=v[1], leader=v[2])
    return v, seed, r["fnd"], r["hnd"], r["lnd"], r["hops"]


def run_jobs(jobs, workers):
    if workers <= 1:
        return [job(j) for j in jobs]
    with ProcessPoolExecutor(max_workers=workers) as ex:
        return list(ex.map(job, jobs, chunksize=4))


def collect(rows, seeds):
    out = {v: np.zeros((len(seeds), 4)) for v in VARIANTS}
    pos = {s: i for i, s in enumerate(seeds)}
    for v, s, f, h, l, hp in rows:
        out[v][pos[s]] = (f, h, l, hp)
    return out


def ci95(x):
    return 0.0 if len(x) < 2 else float(stats.t.ppf(0.975, len(x) - 1) * np.std(x, ddof=1) / np.sqrt(len(x)))


def effects(res):
    """Main effect of each switch: change of the metric when only that switch is turned on, averaged over the others."""
    rows = []
    for fi, (fname, off, on) in enumerate((("2-opt chain (vs greedy)", "greedy", "2opt"), ("cut long links (vs no cut)", False, True),
                                           ("energy-aware leader (vs rotate)", "rotate", "energy"))):
        for c, m in enumerate(("FND", "HND", "LND")):
            a = np.mean([res[v][:, c] for v in VARIANTS if v[fi] == off], axis=0)
            b = np.mean([res[v][:, c] for v in VARIANTS if v[fi] == on], axis=0)
            p = stats.wilcoxon(a, b).pvalue if np.any(a != b) and len(a) > 5 else float("nan")
            rows.append((fname, m, 100 * (b.mean() / a.mean() - 1), p))
    return rows


def fmt_p(p):
    return "n/a" if p != p else ("<0.001" if p < 1e-3 else f"{p:.3f}")


def realism(workers_unused=None):
    """Real layout: how do ARQ retries and idle/sleep energy change the ranking?"""
    L = json.load(open(REAL))
    pos = np.array(L["positions"])
    base = replace(Params(), n_nodes=len(pos), field=float(L["field"]), sink=tuple(L["sink"]))
    rows = []
    for att, slp in itertools.product((1.0, 1.5, 2.0), (0.0, 1e-4, 5e-4)):
        P = replace(base, attempts=att, e_sleep=slp)
        for name in ("LEACH", "PEGASIS", "PEGASIS-2opt", "PEGASIS-E", "PEGASIS-MC"):
            seeds = range(10) if name == "LEACH" else [0]
            m = np.mean([[PROTOCOLS[name](P, pos, np.random.default_rng(10_000 + s))[k] for k in ("fnd", "hnd", "lnd")]
                         for s in seeds], axis=0)
            rows.append((att, slp, name, *m))
    return rows


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--workers", type=int, default=os.cpu_count() or 1)
    ap.add_argument("--seeds", type=int, default=30)
    a = ap.parse_args()
    seeds = list(range(a.seeds))
    md = ["# Ablation study (PEGASIS family)", "", f"Workers: {a.workers}. FND/HND/LND = round of first / half / last node death.", "
      "]

    res = collect(run_jobs([(v, "random", s) for v in VARIANTS for s in seeds], a.workers), seeds)
    tex = ["\\begin{tabular}{lcccc}", "\\toprule", "Variant (chain/cut/leader) & FND & HND & LND & Hops/round \\\\", "\\midrule"]
    md += [f"## A. {a.seeds} random layouts (80 nodes, 400 m block), mean +- 95% CI", "",
           "| Variant (chain/cut/leader) | FND | HND | LND | hops/round |", "|---|---|---|---|---|"]
    for v in VARIANTS:
        x = res[v]
        cells = [f"{x[:, c].mean():.0f} \u00b1 {ci95(x[:, c]):.0f}" for c in range(3)] + [f"{x[:, 3].mean():.1f}"]
        md.append(f"| {vname(v)}{'  (= original PEGASIS)' if v == ('greedy', False, 'rotate') else ''} | " + " | ".join(cells) + " |")

        tex.append(vname(v) + " & " + " & ".join(c.replace("\u00b1", "$\\pm$") for c in cells) + " \\\\")
    tex += ["\\bottomrule", "\\end{tabular}"]
    open(os.path.join(HERE, "ablation_table.tex"), "w").write("\n".join(tex) + "\n")

    eff = effects(res)
    md += ["", "### Main effect of each switch (average change over the other two switches; paired Wilcoxon over layouts)", "",
           "| Switch turned on | Metric | Change | p |", "|---|---|---|---|"]
    etex = ["\\begin{tabular}{llcc}", "\\toprule", "Switch turned on & Metric & Change & $p$ (Wilcoxon) \\\\", "\\midrule"]
```

```
    for f, m, ch, p in eff:
        md.append(f"| {f} | {m} | {ch:+.1f}% | {fmt_p(p)} |")
        etex.append(f"{f} & {m} & {ch:+.1f}\\% & {fmt_p(p).replace('<', '$<$')} \\\\")
    etex += ["\\bottomrule", "\\end{tabular}"]
    open(os.path.join(HERE, "ablation_effects.tex"), "w").write("\n".join(etex) + "\n")

    if os.path.exists(REAL):
        rr = collect(run_jobs([(v, "real", 0) for v in VARIANTS], 1), [0])
        md += ["", "## B. REAL layout (80 OpenStreetMap buildings) - deterministic, one run each", "",
               "| Variant | FND | HND | LND | hops/round |", "|---|---|---|---|---|"]
        for v in VARIANTS:
            x = rr[v][0]
            md.append(f"| {vname(v)} | {x[0]:.0f} | {x[1]:.0f} | {x[2]:.0f} | {x[3]:.1f} |")
        rl = realism()
        md += ["", "# Realism sensitivity on the REAL layout", "",
               "attempts = mean transmissions per packet (ARQ retries); sleep = idle energy per node per round (J; E0 = 0.5 J).",
               "LEACH: mean of 10 runs; others deterministic.", "",
               "| attempts | sleep (J/round) | Protocol | FND | HND | LND |", "|---|---|---|---|---|---|"]
        rt = ["\\begin{tabular}{ccl rrr}", "\\toprule", "Attempts & Sleep (J/round) & Protocol & FND & HND & LND \\\\", "\\midrule"]
        for att, slp, name, f, h, l in rl:
            md.append(f"| {att} | {slp:g} | {name} | {f:.0f} | {h:.0f} | {l:.0f} |")
            rt.append(f"{att} & {slp:g} & {name} & {f:.0f} & {h:.0f} & {l:.0f} \\\\")
        rt += ["\\bottomrule", "\\end{tabular}"]
        open(os.path.join(HERE, "realism_table.tex"), "w").write("\n".join(rt) + "\n")
    open(os.path.join(HERE, "ablation_results.md"), "w").write("\n".join(md))
    print("\n".join(md))


if __name__ == "__main__":
    main()
```

A.4 `code/store.py` – SQLite experiment store

```
"""
store.py -- SQLite experiment store (standard library only).
Every simulated run that the API computes is saved, so results survive restarts and can be listed / compared later.

  runs(id, key UNIQUE, created, protocol, layout, seed, n, rounds, fnd, hnd, lnd, avg_hops, sink_packets, readings)
  curves(run_id -> runs.id ON DELETE CASCADE, alive JSON, energy JSON)

All SQL uses ? placeholders (parameterised queries), so user input can never change the SQL statement.
"""
import json, os, sqlite3, time
from contextlib import closing

HERE = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.environ.get("WSN_DB", os.path.join(HERE, "wsn.db"))
SCHEMA = """
CREATE TABLE IF NOT EXISTS runs(
  id INTEGER PRIMARY KEY AUTOINCREMENT, key TEXT UNIQUE NOT NULL, created TEXT NOT NULL,
  protocol TEXT NOT NULL, layout TEXT NOT NULL, seed INTEGER NOT NULL, n INTEGER NOT NULL, rounds INTEGER,
  fnd INTEGER, hnd INTEGER, lnd INTEGER, avg_hops REAL, sink_packets INTEGER, readings INTEGER);
CREATE TABLE IF NOT EXISTS curves(
  run_id INTEGER PRIMARY KEY REFERENCES runs(id) ON DELETE CASCADE, alive TEXT NOT NULL, energy TEXT NOT NULL);
"""


def connect():
    c = sqlite3.connect(DB_PATH)
    c.row_factory = sqlite3.Row
    c.execute("PRAGMA foreign_keys = ON")
    c.executescript(SCHEMA)
    return c


def save_run(s: dict, layout: str) -> int:
    """Insert (or replace) one run summary produced by platform_core.summary()."""
    key = f"{s['protocol']}|{layout}|{s['seed']}|{s['n']}"
    with closing(connect()) as c, c:
        c.execute("DELETE FROM runs WHERE key = ?", (key,))          # replacing also deletes the old curve (CASCADE)
        cur = c.execute(
            "INSERT INTO runs(key, created, protocol, layout, seed, n, rounds, fnd, hnd, lnd, avg_hops, sink_packets, readings)"
            " VALUES (?,?,?,?,?,?,?,?,?,?,?,?)",
            (key, time.strftime("%Y-%m-%d %H:%M:%S"), s["protocol"], layout, s["seed"], s["n"], s["rounds"],
             s["fnd"], s["hnd"], s["lnd"], s["avg_hops"], s["sink_packets"], s["readings"]))
        c.execute("INSERT INTO curves(run_id, alive, energy) VALUES (?,?,?)",
                  (cur.lastrowid, json.dumps(s["alive_curve"]), json.dumps(s["energy_curve"])))
        return cur.lastrowid


def history(limit: int = 20, protocol: str | None = None) -> list:
    sql = "SELECT id, created, protocol, layout, seed, n, rounds, fnd, hnd, lnd, avg_hops FROM runs"
    args: tuple = ()
    if protocol:
        sql += " WHERE protocol = ?"
        args = (protocol,)
    sql += " ORDER BY id DESC LIMIT ?"
    with closing(connect()) as c:
        return [dict(r) for r in c.execute(sql, args + (int(limit),))]
```

```

def curve(run_id: int):
    with closing(connect()) as c:
        r = c.execute("SELECT r.protocol, r.layout, r.seed, r.n, c.alive, c.energy FROM runs r JOIN curves c ON c.run_id = r.id"
                      " WHERE r.id = ?", (int(run_id),)).fetchone()
    if r is None:
        return None
    return dict(id=int(run_id), protocol=r["protocol"], layout=r["layout"], seed=r["seed"], n=r["n"],
                alive=json.loads(r["alive"]), energy=json.loads(r["energy"]))


def count() -> int:
    with closing(connect()) as c:
        return c.execute("SELECT COUNT(*) FROM runs").fetchone()[0]
```

The remaining platform files (`platform_core.py`, `server.py`, `prepare_layout.py`, `benchmark.py`, `make_figures.py`, the dashboard, the Node-RED flow and the tests) are in the project archive.

B. REALISM SENSITIVITY TABLE

C. QUICK REPRODUCTION CHECKLIST

1. Install Python $\geq$3.10 and `pip install numpy scipy matplotlib`.
2. `cd code && python wsn_sim.py` $\rightarrow$ all ten checks must print `[PASS]`.
3. `python run_experiments.py` $\rightarrow$ regenerates `results/` and `figures/`.
4. Upload `main.tex`, `code/`, `figures/`, `results/` to Overleaf; compiler: pdfLaTeX; press Recompile.
5. To use real data, edit `make_layout` in `wsn_sim.py` to read your surveyed coordinates and change `Params.sink`.

Table 15. Real map: effect of retries (attempts) and idle energy (sleep, J per round). LEACH: mean of 10 runs.

Attempts	Sleep (J/round)	Protocol	FND	HND	LND
1.0	0	LEACH	17	66	392
1.0	0	PEGASIS	4	339	799
1.0	0	PEGASIS-2opt	160	493	975
1.0	0	PEGASIS-E	677	925	945
1.0	0	PEGASIS-MC	677	839	886
1.0	0.0001	LEACH	17	66	339
1.0	0.0001	PEGASIS	4	308	706
1.0	0.0001	PEGASIS-2opt	157	448	820
1.0	0.0001	PEGASIS-E	596	790	809
1.0	0.0001	PEGASIS-MC	596	719	758
1.0	0.0005	LEACH	17	64	232
1.0	0.0005	PEGASIS	4	257	473
1.0	0.0005	PEGASIS-2opt	120	356	501
1.0	0.0005	PEGASIS-E	404	492	506
1.0	0.0005	PEGASIS-MC	404	474	492
1.5	0	LEACH	11	45	261
1.5	0	PEGASIS	3	227	532
1.5	0	PEGASIS-2opt	99	326	665
1.5	0	PEGASIS-E	460	630	653
1.5	0	PEGASIS-MC	460	543	609
1.5	0.0001	LEACH	11	45	237
1.5	0.0001	PEGASIS	3	212	468
1.5	0.0001	PEGASIS-2opt	99	315	594
1.5	0.0001	PEGASIS-E	421	565	585
1.5	0.0001	PEGASIS-MC	421	537	550
1.5	0.0005	LEACH	10	44	180
1.5	0.0005	PEGASIS	3	184	399
1.5	0.0005	PEGASIS-2opt	80	265	407
1.5	0.0005	PEGASIS-E	315	395	411
1.5	0.0005	PEGASIS-MC	315	374	385
2.0	0	LEACH	8	34	203
2.0	0	PEGASIS	2	157	417
2.0	0	PEGASIS-2opt	80	252	500
2.0	0	PEGASIS-E	348	478	499
2.0	0	PEGASIS-MC	348	450	460
2.0	0.0001	LEACH	8	33	191
2.0	0.0001	PEGASIS	2	167	450
2.0	0.0001	PEGASIS-2opt	80	255	458
2.0	0.0001	PEGASIS-E	325	439	461
2.0	0.0001	PEGASIS-MC	325	402	427
2.0	0.0005	LEACH	7	33	149
2.0	0.0005	PEGASIS	2	162	315
2.0	0.0005	PEGASIS-2opt	77	209	342
2.0	0.0005	PEGASIS-E	258	331	346
2.0	0.0005	PEGASIS-MC	258	313	332